Supplement to "FactGate: A Task-Scoped Data Gateway for AI Agents with Cumulative Exposure Accounting"

Minmin Wu

Throughout, FactGate names the system of the main paper. The sealed evaluation contracts, evidence files, and prototype identifiers label the same system `bdg` (an earlier internal name); those labels are digest-pinned and are quoted verbatim where a contract is cited. The formal objects and artifact states retain the names defined in the main paper.

I. Complete Inference Rules

This section contains the full typing and big-step rules summarized in the main paper. Undefined premises reject the plan. Operational SQL join trees are first resolved through the canonical equi-join graph and deterministic binary fold defined in the main paper, so the existing binary T-Join/E-Join rules apply at every fold step. Join fields are stable role-qualified IDs; equality requires the same canonical type and collation; union schemas are identical including type and collation.

A. Canonical Scalar Domain

The closed domain contains `smallint`, `integer`, `bigint`, `numeric`, `real`, `double precision`, `boolean`, `bytea`, `date`, `time without time zone`, both timestamp forms, `uuid`, `text`, `character`, `character varying`, and `jsonb`, with PostgreSQL aliases mapped to these names. Integers retain their widths; exact numerics are reduced rationals; floats retain IEEE width with canonical NaN, infinities, and negative zero. Temporal values use PostgreSQL microseconds and `timestamptz` uses UTC. Text binds deterministic collation name and actual version. Text-preserving `json`, `time with time zone`, and floating-point SUM are excluded. Aggregate results follow PostgreSQL: COUNT returns `int8`; SUM of int2/int4 returns `int8`, int8/numeric returns `numeric`; MIN/MAX retain input type. Unsupported casts, arithmetic, or overflow make evaluation undefined.

B. Typing

$$\frac{D(\nu,s):\Gamma \quad \mathrm{canonical}_\Pi(\Gamma,r)}{\Pi \vdash \mathrm{Scan}[\nu,s,r,\Gamma]:\Gamma}\ (T\text{-Scan})$$

$$\frac{\Pi \vdash e:\Gamma \quad F \subseteq \mathrm{dom}\,\Gamma \quad \Pi;\Gamma \vdash p:\mathbb{B}_3}{\Pi \vdash \sigma_{p;F}(e):\Gamma}\ (T\text{-Select})$$

$$\frac{\Pi \vdash e:\Gamma \quad \varnothing \neq A \subseteq \mathrm{dom}\,\Gamma}{\Pi \vdash \pi_A(e):\Gamma|_A}\ (T\text{-Project})$$

$$\frac{\Pi \vdash e_i:\Gamma_i\ (i=1,2) \quad \mathrm{dom}\,\Gamma_1 \cap \mathrm{dom}\,\Gamma_2 = \varnothing \quad \mathrm{eqtyped}_\Pi(\Theta)}{\Pi \vdash e_1 \bowtie_\Theta e_2 : \Gamma_1 \uplus \Gamma_2}\ (T\text{-Join})$$

$$\frac{\Pi \vdash e_1:\Gamma \quad \Pi \vdash e_2:\Gamma \quad \mathrm{distinctable}_\Pi(\Gamma)}{\Pi \vdash e_1 \cup_\delta e_2 : \Gamma}\ (T\text{-Union})$$

$$\frac{\Pi \vdash e:\Gamma \quad G \subseteq \mathrm{dom}\,\Gamma \quad |G|+|\mathcal{A}|>0 \quad \mathrm{distinctable}_\Pi(G) \quad \Pi;\Gamma \vdash \mathcal{A}:\Gamma_{\mathcal{A}}}{\Pi \vdash \gamma_{G;\mathcal{A}}(e):\Gamma|_G \uplus \Gamma_{\mathcal{A}}}\ (T\text{-Group})$$

$$\frac{\Pi \vdash e:\Gamma \quad o,l \in \mathbb{N} \quad \Pi;\Gamma \vdash O{+}K:\mathrm{TotalOrder}}{\Pi \vdash \mathrm{Page}_{O,K,o,l}(e):\Gamma}\ (T\text{-Page}).$$

C. Output Keys

For $\mathrm{RID}(R,t) = H(\texttt{relation-row}, N(\Gamma), \mathcal{B}, t.k)$, Table I gives every output-key case used by derived FactID. H is domain-separated, typed, and length delimited. For a multi-source operational join, $\mathrm{Fold}_\Pi(G_J)$ fixes the sequence of these binary key constructions independently of SQL alias, join order, and AST parenthesization.

M. Wu is with China Telecom Global Limited. E-mail: wuminmin@chinatelecomglobal.com.

TABLE I
COMPLETE OUTPUT-ROW IDENTITY. EVERY DERIVED FACT ADDITIONALLY BINDS THE DISPLAYED SOURCE/SNAPSHOT BUNDLE $\mathcal{B}$.

Result	Product/snapshot bundle	Canonical output entity key k_o
Scan	$[(\nu, s)]$	Catalog entity key e; online lowering computes $H(\texttt{base-entity}, \nu, (a, \tau, C_\tau(v))_{a\in K})$
Unary/Page	Child bundle	The retained child's k_o
Binary Join fold step (t_1, t_2)	Sorted namespace-wise merge of the two child bundles	$H(\texttt{join-row}, \mathrm{sort}[\mathrm{RID}(R_1, t_1), \mathrm{RID}(R_2, t_2)])$
Union-distinct class Q	Sorted namespace-wise merge of both bundles	$H(\texttt{union-distinct-row}, N(\Gamma), (a, \tau, C_\tau(v_a))_{a\in \mathrm{sort}\,\Gamma})$
Keyed group Q	Child bundle	$H(\texttt{group-row}, \mathrm{sort}(N(x_a), \tau_a, C_{\tau_a}(v_a))_{a\in G})$
Global aggregate	Child bundle	$H(\texttt{group-row}, \texttt{global})$; the Fact also binds aggregate expression, value, bundle, and $CW(W)$

D. Big-Step Evaluation

Write $\alpha_r(t) = (S_r, W_r)$ and $\alpha_a(t) = (S_a, W_a)$. $\mathrm{aug}_F(t)$ replaces $\alpha_r(t)$ by $\alpha_r(t) \oplus \bigoplus_{a\in F} \alpha_a(t)$ and otherwise copies the cell map. A derived cell is $\langle v_a, \tau_a, \alpha_a, x_a, \rho_a\rangle$. Each displayed bag is paired with its statically derived schema and source/snapshot bundle. Every conclusion is followed by the representation validator; failure removes the derivation.

[E-Scan]
$$\frac{D(\nu, s) = [(e_i, v_i)]_i}{\Pi; D \vdash \mathrm{Scan}[\nu, s, r, \Gamma] \Downarrow \{\{\langle e_i, \{a \mapsto \langle v_i[a], \tau_a, \{BC_i^a\}, [BC_i^a \mapsto 1], \nu.a, BC_i^a\rangle\}, \{BR_i\}, [BR_i \mapsto 1], \{(r, \nu, e_i)\}\rangle\}\}_i},$$
$$BR_i = f_{row}(\nu, s, e_i), \quad BC_i^a = f_{cell}(\nu, s, e_i, a, \tau_a, v_i[a]).$$

[E-Select]
$$\frac{\Pi; D \vdash e \Downarrow R}{\Pi; D \vdash \sigma_{p;F}(e) \Downarrow R[\mathrm{rows} := \{\{\mathrm{aug}_F(t) \mid t \in R, [\![p]\!]_{\Pi,t} = \mathsf{T}\}\}]}.$$

[E-Project]
$$\frac{\Pi; D \vdash e \Downarrow R}{\Pi; D \vdash \pi_A(e) \Downarrow R[\Gamma := \Gamma|_A, \mathrm{cells}(t) := \mathrm{cells}(t)|_A]}.$$

[E-Join]
$$\frac{\Pi; D \vdash e_j \Downarrow R_j \ (j = 1, 2)}{\Pi; D \vdash e_1 \bowtie_\Theta e_2 \Downarrow \{\{J(t_1, t_2) \mid t_j \in R_j, \textstyle\bigwedge_{(a=b)\in\Theta} [\![t_1.a = t_2.b]\!]_\Pi = \mathsf{T}\}\}},$$
$$J.k = H(\texttt{join-row}, \mathrm{sort}[RID_1, RID_2]),$$
$$\alpha_r(J) = \alpha_r(t_1) \oplus \alpha_r(t_2) \oplus \bigoplus_{(a=b)\in\Theta} (\alpha_a(t_1) \oplus \alpha_b(t_2)),$$
$$J.\,\mathrm{cells} = t_1.\,\mathrm{cells} \uplus t_2.\,\mathrm{cells}\,.$$

[E-Union]
$$\frac{\Pi; D \vdash e_j \Downarrow R_j \ (j = 1, 2)}{\Pi; D \vdash e_1 \cup_\delta e_2 \Downarrow \{\{U_Q \mid Q \in (R_1 \uplus R_2)/\equiv_\Gamma\}\}},$$
$$U_Q.k = k_{union}(\Gamma, \mathrm{tuple}(Q)), \quad \beta_\Gamma(t) = \alpha_r(t) \oplus \bigoplus_{a\in\Gamma} \alpha_a(t),$$
$$\alpha_r(U_Q) = \bigsqcup_{t\in Q} \beta_\Gamma(t),$$
$$U_Q.\,\mathrm{cells}(a) = \langle v_a, \tau_a, \bigsqcup_{t\in Q} \alpha_a(t), N(\texttt{union}(x_{t,a})), \rho_a\rangle,$$
$$\rho_a = \begin{cases} \mathrm{mat}(U_Q, a), & \text{if every member materializes to one FactID,} \\ \bot, & \text{otherwise.} \end{cases}$$

[E-Group]
$$\frac{\Pi; D \vdash e \Downarrow R}{\Pi; D \vdash \gamma_{G;\mathcal{A}}(e) \Downarrow \{\{G_Q \mid Q \in \mathrm{Part}_G(R)\}\}},$$
$$G_Q.k = k_{group}(G, Q), \quad \alpha_r(G_Q) = \bigoplus_{t\in Q} \left(\alpha_r(t) \oplus \bigoplus_{g\in G} \alpha_g(t)\right),$$
$$G_Q.\,\mathrm{cells}(g) = \langle v_g, \tau_g, \bigoplus_{t\in Q} \alpha_g(t), N(x_g), \mathrm{mat}(G_Q, g)\rangle,$$
$$G_Q.\,\mathrm{cells}(a') = \langle \mathrm{PgAgg}_\Pi(Q, G, \mathcal{A})_{a'}, \tau_{a'}, \bigoplus_{t\in Q} \alpha_{\mathrm{arg}(a')}(t), N(x_{a'}), \mathrm{mat}(G_Q, a')\rangle.$$

[E-Page]
$$\frac{\Pi; D \vdash e \Downarrow R \quad L = \mathrm{sort}_{\prec O,K}(R)}{\Pi; D \vdash \mathrm{Page}_{O,K,o,l}(e) \Downarrow R[\mathrm{rows} := L[b : z]]},$$
$$b = \min(o, n), \quad z = \min(n, o + l^*), \quad l^* = \begin{cases} n, & l = 0, \\ l, & l > 0. \end{cases}$$

At graph-fold step i, e_1 denotes the accumulated component and e_2 the next canonical node. The step predicate Θ_i is the union of every per-edge predicate set connecting that node to a node already in e_1. Connectivity makes Θ_i nonempty, and adding each edge when its later endpoint enters the component consumes every graph edge exactly once.

Here $\equiv_\Gamma$ is componentwise `IS NOT DISTINCT FROM`. Union support includes every candidate and every distinctness field; pointwise maximum combines alternative witnesses. Grouping includes group-key inputs and the complete declared aggregate argument, including NULL and non-extreme `MIN/MAX` inputs. Exact/integer `SUM` is admitted; floating-point `SUM` is undefined. Pagination uses PostgreSQL ordering plus a Catalog key suffix that proves totality, and copies annotations only for the selected output slice. These are positive-output dependencies, not physical-read or minimal causal provenance.

E. Operator Summary

TABLE II
SUMMARY OF THE BIG-STEP EFFECT PER OPERATOR. "ROW ANNOTATION" IS ALWAYS INCLUDED IN THE DEPENDENCY SET OF A SELECTED OUTPUT ROW.

Operator	Output construction	Positive-output dependency-footprint rule
Scan	Keyed PostgreSQL bag; base cells retain typed values	Singleton base-row and base-cell FactIDs
Select/Project	Keep only T rows; restrict visible cells	Add every predicate field to retained row annotations; projection preserves them
Inner join	Every matching child-row pair at one binary fold step under PostgreSQL equality	Union both row annotations and all join-key cell annotations; preserve fanout in witnesses
Union-distinct	Classes under componentwise `IS NOT DISTINCT FROM`	Include every candidate and every distinctness field, including hidden fields; max-merge alternative witnesses
Group/ aggregate	One output per key class; global empty input yields one row	Include group-key inputs and every declared aggregate argument; `COUNT(*)` uses member rows, and NULL/non-extreme inputs remain dependencies
Page	Stable total-order slice; annotations copied	Account selected output rows only; non-output rows and rank are outside the profile; ORDER BY columns must be selected, so only the Catalog tie-break suffix is order-only

F. Extensions: Derived Cells, Exact AVG, COUNT DISTINCT, and HAVING

Four constructs were admitted after the rules above were written (the agent-written workload's first pass rejected them). Their typing and effect rules follow; Table III maps each to its rule, implementation site, test, and evidence version.

Derived cells. An arithmetic term is $x ::= a \mid n \mid x \oplus x$ with $\oplus \in \{+, -, \times\}$, a a column of the single Product in scope, and n an integer literal. Division, casts, functions, and multi-Product terms are undefined.

$$\frac{\Pi \vdash e : \Gamma \quad |\mathcal{B}(e)| = 1 \quad \forall a \in \mathrm{cols}(x) : \Gamma(a) = \tau \quad \tau \in \{\texttt{int2}, \texttt{int4}, \texttt{int8}, \texttt{numeric}\}}{\Pi \vdash \pi_{A;\, x:\tau}(e) : \Gamma|_A \uplus \{x \mapsto \tau\}} \ (T\text{-Derive})$$

$$\frac{\Pi; D \vdash e \Downarrow R}{\Pi; D \vdash \pi_{A;x}(e) \Downarrow R'} \ (E\text{-Derive})$$
$$R' = R[\Gamma := \Gamma|_A \uplus \{x\},\ \mathrm{cells}(t) := \mathrm{cells}(t)|_A \uplus \{x \mapsto d_x(t)\}],$$
$$d_x(t) = \langle [\![x]\!]_{\Pi,t}, \tau, \textstyle\bigoplus_{a \in \mathrm{cols}(x)} \alpha_a(t), N_{\mathrm{arith}}(x), \mathrm{mat}(R, t, x)\rangle.$$

Integer literals adopt τ; the value is PostgreSQL's under exact arithmetic (NULL propagates; overflow makes evaluation undefined and settles nothing). N_{arith} sorts the operands of $+$ and $\times$ by stable identity, keeps the order of $-$, canonicalizes literals under τ, and erases aliases, so the derived Release identity $f_d = \langle \varpi, \mathcal{B}, k_o, N_{\mathrm{arith}}(x), \tau, C_\tau(v), CW(W)\rangle$ is the main text's derived identity with $N_\varpi := N_{\mathrm{arith}}$. The Dependency footprint of the cell is the union of its argument cells' supports; the row's support is unchanged. Split/replay conservation is the derived-identity case of the base-ledger proposition: the same term over the same row yields the same f_d in any spelling.

Exact AVG. $\mathcal{A}$ gains `AVG` over $\tau \in \{\texttt{int2}, \texttt{int4}, \texttt{int8}, \texttt{numeric}\}$ only, with output type `numeric`. PostgreSQL computes it as one exact `numeric` division of the exact sum by the count, so the value is fold-order independent; floating-point `AVG` is excluded for the same reason as floating-point `SUM`. Its E-Group clause is that of `SUM`: the aggregate cell's annotation is the complete argument bag of the group, and its identity binds $N(\texttt{avg}(a))$, the value, and $CW(W)$.

COUNT DISTINCT. `COUNT(DISTINCT` a`)` is admitted; `DISTINCT` under any other aggregate is undefined. PostgreSQL's value ignores NULL and repeated values, but the Dependency rule is that of `COUNT(`a`)`: every member row's argument cell, NULL and duplicates included, since each member positively contributed to the group. The result identity binds the `DISTINCT` flag inside $N(x_{a'})$, so `COUNT(`a`)` and `COUNT(DISTINCT` a`)` over the same group are distinct Release facts and distinct Outcome composites.

HAVING. A `HAVING` clause is $\sigma_{h;F_h}$ applied to the output of $\gamma_{G;\mathcal{A}}$, typed by T-Select with $\Gamma = \Gamma|_G \uplus \Gamma_{\mathcal{A}}$ and h ranging over aggregate output aliases with typed literals only; its E-Select clause keeps the group rows whose h is T and adds each read aggregate cell's annotation (already the complete argument bag) to the surviving row's annotation. Groups `HAVING` drops are non-output and contribute nothing (positive-output rule); PostgreSQL returns no row for them, and the production evaluator reconstructs group membership only for returned rows. `HAVING` predicates enter the typed normal form and the plan digest, hence the replay key and the Outcome composite.

TABLE III
EXTENSION CONSTRUCTS: RULE, IMPLEMENTATION SITE, TEST, AND EVIDENCE VERSION. EVIDENCE COMMIT `4BC1652EC43F`; THE GENERATED-PLAN CAMPAIGN IS SEED 20260830, 5,000 PLANS, DIGEST `79E25395CB4B`.

Construct	Rule	Implementation	Test and evidence
Derived cell $\{+, -, \times\}$	T/E-Derive; N_{arith}	`internal/sqllowering/lower_derived.go` (single-Product, exact type, no cast); `internal/exposure/algebra_v2.go` derived fact; `internal/queryplan` normal form	lowering fragment tests; generated-plan differential campaign (Section XXVIII); agent-written workload (Section XXXI)
Exact `AVG`	E-Group (`SUM` clause), output `numeric`	`internal/exposure/algebra_v2.go` aggregate admission and result type; `internal/queryplan/normalform.go`	generated-plan campaign (exact `avg` plans); RQ1 oracle memberships
`COUNT(DISTINCT)`	E-Group (`COUNT` clause); identity binds the flag	`internal/exposure/algebra_v2.go` `AggregateSpecV2.Distinct`	generated-plan campaign (`count(distinct)` plans)
`HAVING`	σ over γ output; E-Select	`internal/exposure/algebra_v2.go` having-aware group reconstruction; `internal/queryplan/normalform.go` `Having`	generated-plan campaign (`HAVING` plans); benign-trace pilot (Section XXIII-C), which found and fixed the identity-list defect

II. RESTRICTED SEMANTIC-VIEW FRONT END

A. Registry and Expansion Boundary

Let $\mathcal{R}$ be a finite registry keyed by canonical schema-qualified relation names. A record contains a kind, ordered column contract, and either (i) an ordinary-View definition and sorted direct dependencies, or (ii) the Catalog Product identity of an opaque terminal publication. Discovery reads `pg_class`, `pg_rewrite`, `pg_depend`, `pg_attribute`, type/collation metadata, and exact `pg_get_viewdef` output in one read-only `REPEATABLE READ` transaction. It also parses the definition independently; the parsed relation set must equal the catalog-derived dependency set. This second path exposes, and then rejects, recursive self references and unsupported built-in/object dependencies that a relation-only dependency walk could otherwise miss.

DFS colors each ordinary View white/gray/black, so a gray back-edge rejects as a cycle. Ordinary Views recurse; a populated materialized View is a leaf only when mapped to an authorized Product. Raw/partitioned/foreign/system relations, temporary or nonpersistent relations, row-security/options, unpopulated materializations, and volatile, user-defined, or otherwise unsupported object dependencies reject. Stable names, exact definitions, ordered columns, dependency edges, and PostgreSQL major version form the registry revision; transaction-local OIDs are not serialized.

For a successfully parsed View v, expansion computes a fragment

$$X(v) = \langle S, \Theta, F, G, A, O, b \rangle,$$

where S is the occurrence-preserving sequence of governed base scans, Θ the typed equality predicates, F the typed literal filters, G the group fields, A the aggregate bindings, O the ordered public outputs, and b records whether an aggregate barrier has been crossed. A relation reference recursively obtains and clones its child fragment; alias-qualified columns are substituted by the child's stable field bindings. INNER JOIN concatenates child occurrences and equalities, selection appends conjuncts, and direct projection substitutes output bindings. A grouped definition sets b and binds only direct group fields plus approved `COUNT(*)`/`COUNT(field)`/`SUM`/`MIN`/`MAX` expressions. If any child already has $b = 1$, the parent must have exactly that one input and be only a direct projection/rename; a join, filter, group, or new aggregate is undefined.

The source grammar additionally excludes `DISTINCT`, all set operations, CTEs/subqueries, `HAVING`, windows, row locking, ordering/paging, projection expressions and casts, non-AND conditions, and non-equality join conditions. Literal casts emitted by PostgreSQL's deparser are allowed only when the cast's canonical type equals the resolved field type. The expanded fragment must contain a connected equi-join graph and at most 16 distinct Product/stable-role scans; a shared DAG child is not silently deduplicated, so two occurrences that resolve to the same stable role reject. The implementation also caps View depth at 16, reachable nodes at 64, dependency edges at 128, expanded predicates at 128, and reachable definition text at 1 MiB.

After these checks, the compiler constructs the same canonical G_J used for direct SQL, emits existing `JoinMany`, attaches F, G, A, O, and invokes the shared typed normalizer. Hence transparent nesting is eliminated before the accounting semantics. This is a constructive mapping for the accepted AST fragment. It is neither a theorem about PostgreSQL's full rewrite system nor a semantic-equivalence decision procedure for rejected SQL.

At query time the composer accepts an outer plan naming one semantic root and uses the artifact's ordered output bindings to replace public names by stable FieldIDs. For an ungrouped artifact it composes supported projection, literal filtering, grouping, and aggregation with the expanded input. For an artifact that has crossed the aggregate barrier, only projection/reordering of its already computed outputs is defined. A query-time join between the semantic root and another Product, and `ORDER BY`, `LIMIT`, or `OFFSET` above the root, are undefined in the current profile. These restrictions are above the root: they do not reduce the arbitrary connected topology or multi-equality edges accepted inside a closure of at most 16 sources/depth 16. The resulting plan is passed to the existing `CompileRelational` path, so its visible statement, provenance program, normal form, and terminal ordinal bindings share the non-View implementation.

B. Four Digests and Task Binding

Table IV separates exact change evidence from the expanded semantic identity. Every hash is SHA-256 over canonical, domain-separated input.

TABLE IV
SEMANTIC-VIEW CONTRACT DIGESTS.

Digest	Committed payload
Definition	Exact root `pg_get_viewdef` bytes; no whitespace normalization
Dependency	Sorted reachable nodes/edges, exact per-node definitions, compiled interfaces, kinds/Product leaves, and PostgreSQL major version
Canonical plan	Shared typed query normal form of the fully expanded base-relation plan; syntactic View layers, relation aliases, and accepted join order are absent
Interface	Ordered public output name, canonical SQL type, collation name, and collation version

The Catalog stores and compares all four. A task-set binding then sorts every originally requested Product contract of the form

$$(product, h_{plan}, h_{dep}, h_{iface})$$

and hashes the canonical array under a separate domain. An ordinary non-View Product in the same request receives an opaque contract over its stable Product/publication/interface metadata, so authorization narrowing cannot introduce an unbound Product. The root exact definition remains separately checked and the dependency digest already commits every exact closure definition. Control stores the canonical bytes content-addressed, the binding digest on Grant and query record, and a reverse index from each task/Product to every reachable schema-qualified dependency.

Task binding begins `ACTIVE`. Any current compile failure or mismatch of the persisted set makes `REQUIRE_REBIND` durable; database constraints prevent a transition back to `ACTIVE`. Exact terminal replay is looked up before this new-work gate. Otherwise the enforcement layer re-resolves the current set, reserves ordinary resources, and the connector rechecks the registry revision inside the same Business-PG snapshot used by visible and companion SQL. A connector mismatch releases the reservation and settles no exposure. Rebind creates a new root request and external approval from the old Grant's final narrowed scope; no Grant, settled query, result, or receipt is rewritten.

The signed Grant remains at the semantic-root interface; expanded terminal Products are not silently added to its approved set. For each query the enforcement layer derives an ephemeral least-privilege policy grant for every terminal named by the bound artifact. Public scope outputs are resolved through `Artifact.Output.FieldID` to stable terminal role/column pairs, and every mandatory scope declared by every terminal Product must be covered by the approved root predicate. Missing, ambiguous, or conflicting mappings reject before reservation. The same terminal Products supply their immutable publication/sidecar bindings. The derived grants are neither signed as new authority nor persisted or returned to the client.

C. Compiler and Drift Test Evidence

Five independent, fixed-seed `testing/quick` properties each execute 64 generated cases in the restricted grammar. Each generator constructs a connected graph of two to five governed relations, forces at least one two-equality edge, varies traversal priorities and aliases, and chooses one to three transparent wrappers for nested cases. The properties separately check: (i) complete-artifact determinism across repeated calls and registry/Product insertion orders; (ii) alias invariance; (iii) connected join-order invariance; (iv) direct-versus-nested equivalence; and (v) digest sensitivity that distinguishes semantic, public-interface, and transitive-dependency drift. Boundary tests additionally cover cycles, parsed/catalog dependency mismatch, schema mismatch, safe literal casts, forbidden projection casts, occurrence- preserving shared DAG expansion, depth 16/17, and source count 16/17.

Separate PostgreSQL integration tests discover a three-level View chain ending at a materialized publication, verify transitive child replacement changes the registry revision while the root definition remains equal, reject unsafe and recursive definitions, and observe `repeatable read` on the query path. These tests support deterministic implementation behavior and fail-closed change detection. They are not a generated proof for every grammar term, a formal refinement of PostgreSQL parsing, a multi-source View-DAG performance study, or part of the digest-bound RQ counts in the main paper.

III. Proofs of Main-Text Results

The theorem and property names below match the labeled statements in the main paper. References to effect extraction, annotation invariants, row identity, and outcome identity denote the corresponding displayed main-paper definitions; the complete operator rules they use appear above. These are paper-level, representation-independent arguments over the declared algebra. They do not prove the SQL parser, complete AST, PostgreSQL row execution, cryptographic primitives, network, or ciphertext implementation.

Proof of Fact identity. Each scalar encoder is a canonical representative of its typed equivalence class: bounded integers use signed decimal, exact numerics reduced rationals, floats fixed-width hexadecimal with canonical NaN and zero, temporal values PostgreSQL microseconds (UTC for `timestamptz`), and NULL a disjoint token. Every tuple component is type tagged and length delimited; every list is count prefixed and sorted where its mathematical carrier is a set. Decoding first recovers the unique tag and component boundaries, then the unique scalar representatives. Therefore equal encodings imply equal typed payloads, and determinism follows from the unique representative and ordering functions. □

Proof of Outcome separation and replay. Every Outcome atom payload length-prefixes the predicate context, stable role and FieldID, resolved expression, type/collation, operator, and canonical literal. Changing any component changes the payload and, under the collision assumption, its FactID. The composite length-prefixes the normal-form and result-observation digests, row count, context, complete atom-set digest, and atom cardinality. Exact replay or a supported rewrite makes all these components equal; changing the normal form, result observation, or complete atom set changes the composite payload. Exact set difference therefore charges each novel atom/composite once and charges replay zero. □

Proof of Representation closure and partial-function semantics. For an operational join graph, first use induction over the deterministic fold: connectedness gives the next node at each step, and its nonempty Θ_i contains all predicates on edges from that node to the accumulated component, so every edge is consumed exactly once. Then proceed by structural induction on the resulting e. Scan is fixed by D and constructs singleton support/witness pairs. Select's three-valued predicate and dependency set are fixed by Π; Select, Project, and Page only take a deterministic subbag or copy annotations. Each binary Join enumerates a uniquely determined bag of matching child-row pairs; disjoint schemas and the canonical pair key make its output usable by the next fold step. Union first collapses an identical annotated branch only under the duplicate-free full-tuple premise, then has unique typed-tuple equivalence classes and uses deterministic same-proof union/addition within a candidate and alternative union/max across candidates; E-Union constructs each output cell $\langle v_a, \tau_a, \bigsqcup \alpha_a, N(\texttt{union}(x)), \rho_a \rangle$ from those classes. Group has unique not-distinct classes, the explicitly defined empty class, deterministic exact/numeric sum folds (floating-point SUM is excluded, so no order-dependent fold arises), and one complete PostgreSQL result per class. E-Group composes every group-key input into the row annotation and constructs both group-key and aggregate cells from that result. Set union with multiset sum or max preserves equal support. Total order makes Page's slice unique. Each case constructs the annotation invariant, and validation turns any range, collision, schema, oracle, or multiplicity violation into no derivation. □

Proof of Dependency determinism. Representation closure gives a unique R. Effect extraction ranges over its unique keyed occurrences and fields. C_τ, expression normalization, bundle/key construction, witness commitment, Fact materialization, and final set normalization are deterministic functions. Therefore both finite Fact sets, and hence their ordered canonical encodings, are unique. □

Proof of Group-key completeness. E-Group includes $\bigoplus_{g \in G} \alpha_g(t)$ in the output row annotation. Projection preserves that annotation, and effect extraction includes every selected output row's S_r independently of V. □

Proof of Distinct-dependency completeness. E-Union first composes every schema-field annotation into $\beta_\Gamma(t)$, then alternative-composes all candidates into the output row annotation. Alternative max changes multiplicity but its support is the union, which effect extraction includes. □

Proof of Conservative aggregate input. This is the aggregate-cell clause of E-Group followed by effect extraction. PostgreSQL's scalar fold may ignore NULL or retain only an extreme value, but the dependency rule deliberately ranges over the complete argument bag. □

Proof of Normalization. By Fact identity well-definedness, a validated canonical payload has one map key; a same-digest unequal payload aborts rather than merges. Normalization is therefore finite-set insertion followed by lexical canonical-payload order. Idempotence and commutativity follow from $A \cup A = A$ and $A \cup B = B \cup A$; the fixed serialization order gives the same bytes in each case. □

Proof of Restricted projection/selection invariance. Because $F \subseteq A$, both selection predicates read identical typed cells and retain exactly the rows whose truth value is T. By E-Select each retained row receives the same $\alpha_r(t) \oplus \bigoplus_{a \in F} \alpha_a(t)$; by E-Project both retain the same keys and cells in A. Effect extraction over $V \subseteq A$ therefore yields identical release and dependency sets. Field traversal order is absent from Fact payloads and removed by set normalization. □

Proof of Typed normal-form soundness. The SQL front end first resolves relation aliases to Catalog stable roles, so caller-chosen aliases are absent from graph nodes, fields, and derived expressions. Canonical graph construction sorts nodes by Catalog semantic key, orients each endpoint pair by node rank, coalesces predicates for that pair, and sorts edges and their stable field-pair predicates. The deterministic fold then chooses the first not-yet-joined node adjacent to the accumulated component and attaches every equality from that node to the component. Consequently any accepted INNER JOIN AST with the same connected graph produces the identical binary term, regardless of source order, equality direction/order, or parenthesization. This is not a reassociation rule for arbitrary SQL: it applies only after the AST has passed the equi-join graph boundary.

The remaining typed normalizer can erase only derived display names, conjunction/set traversal order, binary join operand and predicate order, union operand order, an identical duplicate union branch whose child is proven full-tuple distinct, and alternate spelling of the same stable page suffix. A Scan carries no such distinctness proof; Union and Group produce it, Select/Page preserve it, and Project preserves it only for an unchanged schema. Set normalization removes traversal order. Each folded Join enumerates the same matching bag, hashes sorted child identities, and uses commutative and associative $\cup/\oplus$. Union classes are operand independent; same-proof member composition includes all equivalence fields, while alternative max-merge is commutative and idempotent. An identical materialized fact is retained, and relation-level union expressions are sorted before downstream composition. Group keys and witness commitments are sorted. The SQL compiler emits the same Catalog-canonical page suffix represented by the normal form. Every accepted graph spelling therefore denotes the same folded term. Per rule (main-text R1–R6), the effect-preservation lemma and its side condition are: R1 (alias erasure) changes no Fact identity because identities are built from stable roles and canonical fields, never display names; R2 (canonical spellings and typed literals) preserves the evaluated predicate under the attested type, collation, and numeric mode; R3 (sorting commutative components) preserves the bag because AND, projection sets, aggregate lists, group-key sets, and union/join operand order are commutative in the algebra; R4 (join-graph fold) applies only after the equi-join graph boundary, where every spelling of one connected graph enumerates the same matching bag; R5 (duplicate union branch collapse) requires the child's full-tuple distinctness proof, since without it the duplicate branch changes witness multiplicity (Scan carries no proof; Union and Group produce one; Select and Page preserve it; Project preserves it only when the schema is unchanged, because a narrowing projection can merge distinct tuples); R6 (page suffix) preserves the total order the Catalog keys define. Structural induction over well-typed contexts proves the result. □

Proof of Base-ledger split/page/join conservation. For $V = V_A \cup V_B$, distribute both unions in effect extraction over V_A, V_B; this gives $\mathcal{E}^{ri}(R, V) = \mathrm{merge}(\mathcal{E}^{ri}(R, V_A), \mathcal{E}^{ri}(R, V_B))$. E-Page copies each selected keyed row and cell, so a complete disjoint slice partition distributes the same two unions over the row index set. Overlap and repetition disappear by set idempotence. E-Join adds witness multiplicities with $\oplus$, but effect extraction takes their support sets; repeated support membership consequently has dependency cardinality one. A changed derived witness may still change its release Fact, as required. Induction over the canonical fold length applies the same support-union argument to every node in an admitted join graph. □

Proposition 1 (Exact bitmap refinement)**.** *For canonical bitmaps A and B in one dictionary segment,* $\mathrm{Decode}(A \,\mathrm{OR}\, B) = \mathrm{Decode}(A) \cup \mathrm{Decode}(B)$*,* $\mathrm{Decode}(A \,\mathrm{ANDNOT}\, B) = \mathrm{Decode}(A) \setminus \mathrm{Decode}(B)$*, and* $\mathrm{popcount}(A) = |\mathrm{Decode}(A)|$*. For every successful query in the closed online algebra, decoding the streamed bitmap components yields exactly the release and dependency components of $\mathcal{E}(q, D_s)$.*

Proof of Exact bitmap refinement. The first three equalities follow pointwise from the dictionary bijection and exact bitmap membership. For query derivation, structural induction follows the same Scan, Select, Project, Join, Union, Group, and Page cases as effect extraction. Handles substitute singleton base FactIDs; bitmap union substitutes support-set union; sparse maps preserve witness addition/max; and hash-ordered merging preserves the canonical witness commitment. Every case therefore decodes to the oracle FactSet, while any missing premise aborts. For JoinMany, induction over the deterministic fold replaces a two-source handle pair by one handle per source in the companion tuple without changing these pointwise arguments. □

Proof of Atomic root-family budget safety. Every descendant resolves to the same root head. Exact bitmap refinement for Release/dependency and exact Merkle-radix refinement for Outcome make each concrete candidate decode to $K_d \cup E_d$ and its charged count to $|E_d \setminus K_d|$. A successful epoch CAS linearizes all three candidate heads at one point; a conflicting transaction recomputes rather than publishing a stale difference. The precondition checks every Catalog bound and failure publishes no head. Induction over successful CAS order establishes the modeled union identity and all three bounds. The bounded TLA+ model separately checks instances of this abstract transition. Correspondence to the physical implementation is supported by executable evidence rather than by a mechanized refinement proof. □

IV. Outcome Set and Radix Refinement

The current Outcome effect is $A(q) \cup \{c_q\}$. The atomizer expands `IN` to typed equality atoms and `NOT IN` to typed inequality atoms, removes exact duplicates, and binds every atom to the predicate context. The composite binds the canonical normal form, result observation, row count, context, sorted atom set digest, and atom count. Signed limits bound raw literals, unique atoms, one payload, and total payload before Business SQL.

The exact SHA-256 set partitions members by prefix16 into deterministic 4096-member chunks. A prefix8 block commits ordered leaf references, and the root commits at most 256 block references and total cardinality. For a candidate, settlement reads only its prefix8 blocks and prefix16 leaf families, compares full hashes, rebuilds only affected leaf families/blocks, and reuses all other digests. Hence the physical work is $O(k \log k + h + \ell + b)$ for candidate cardinality k, hashes loaded h, leaf chunks ℓ, and blocks b, plus a root manifest bounded by 256 references; the algorithm never scans untouched radix partitions. Its worst-case cost is linear in the membership and manifest references of candidate-touched partitions, which may approach total root size under adversarially concentrated prefixes. The source-controlled PostgreSQL regression commits a 100,000-member object graph before an independent merge transaction; the 8,193-member same-prefix regression covers multi-chunk boundary shifts and missing-chunk failure. These regressions, not the abstract TLA+ model, cover radix partitioning, touched-branch reconstruction, object reuse, and malformed-object rejection.

TABLE V
OUTCOME-SET REPRODUCIBILITY BINDINGS. THE IMPLEMENTATION BASE IDENTIFIES THE CODE LINEAGE; FOR SCHEMA 3, THE BOUNDED SOURCE MANIFEST IS REHASHED FROM THE RECORDED SUBMISSION-COMMIT BLOBS RATHER THAN THE CURRENT WORKING TREE.

Binding	Identity
Implementation base commit	`4bc1652ec43f40416d71ddad50fa524eccc73f51`
Evidence snapshot commit	Schema-3 `submission_commit` in `evaluation/v5-outcome/evidence.json`
Implementation source-set SHA-256	e97638f0f6a2e0183729f36da080e3f8aef2e335e30d35b437f6c1c66249d3b5
Raw `go test -json` SHA-256	5a6217aafb4b9aeeba7578051574e04701024071508f87caf7aa2f27babc22e0
Required tests passing	12; zero skipped and zero failed
Randomized root-family sequences	24 seeds, 92 tasks, 718 settlements (253 committed, 465 refused; 15 seeds reach a limit)

Randomized root-family sequences. The same raw log carries `TestOrdinalRootFamilyRandomSequences ConserveExactNovelty` (`internal/control/ordinal_family_property_test.go`). For each of 24 seeds it approves a root with random limits (6–41 Result, 8–63 Dependency, 3–8 Outcome facts), grows a random delegation tree of up to 6 tasks whose descendants share the root ledger and each sign a ceiling drawn uniformly at or below their parent's in every dimension (67 descendants narrower than their parent in all), and settles up to twenty rounds of random observations (random subsets of a 24-ordinal row segment and a 24-ordinal cell segment, one Outcome fact from a six-payload pool) against the live Control store; one round in three is a concurrent burst of two to four tasks settling at once. The oracle is an ordinary Go map of the committed observations, independent of the bitmap and radix code. After every round the committed root head must equal the map's union in all three dimensions and never exceed the root's ceiling; a settlement is admitted exactly when the family total after it stays within both the root's and the settling task's ceiling (Section 5.1's narrowed-child rule); for sequential steps the head delta must equal the observation's novelty against the union at that moment (zero for replays from any task in the family), and a sequential step is refused exactly when a dimension it adds to would exceed the root's or the settling task's ceiling. Refusals inside a concurrent burst are not re-judged afterwards: a sibling committing later can shrink the refused observation's novelty, so the final union says nothing about the state the refusal was decided against. Over 718 settlements (253 committed, 465 refused) no check failed. The test drives the V4 ordinal settlement engine that V5 Outcome sets build on; it does not exercise the SQL lowering or the companion statement.

V. PUBLICATION EPOCH DESCRIPTOR

A publication epoch is $P_e = \langle datasource, schema, snapshot_bundle, catalog, dictionary \rangle$, where *snapshot_bundle* maps every stable role to its immutable reporting publication, *catalog* binds the bundle's schema, policy, and lineage, and *dictionary* is the set of ordinal dictionaries; h_e is the digest of a canonical descriptor of these five components.

VI. MEASUREMENT HOST

All measurements ran on one WSL2 guest (Ubuntu 22.04.5) of a laptop-class Intel Core i9-13900HX (16 cores, 29 GiB visible), Docker 29.7.2, PostgreSQL 16.14 from a digest-pinned image, and Go 1.25.14; no other workload ran during a campaign.

VII. NUMBER-TO-EVIDENCE MAP

Table VI lists, per research question, the counts the main text cites and the source state of each line.

Table VII maps every group of reported numbers to the commit it was measured at, the retained file it is recomputed from, and the command that regenerates it, and its last column gives the evidence class: sealed campaign, regenerated regression, retained pilot (never publication evidence), simulation, or analysis of retained records. Every macro in the manuscript is emitted by `paper/tkde/generate_evidence.py` (rows above the rule) or `paper/tkde/generate_p10_evidence.py` (rows below it, the studies added after the campaign was sealed); both fail closed when a file, digest, or commit binding disagrees.

TABLE VI
CURRENT SOURCE-BACKED EVIDENCE BY RESEARCH QUESTION: THE COUNTS THE MAIN TEXT CITES AND THE SOURCE STATE OF EACH LINE (TRACKED RESULTS AT THE EVIDENCE COMMIT AND REGENERABLE BY THE LISTED COMMAND, ARCHIVED RUN, OR SEALED PUBLICATION CAMPAIGN).

RQ	Current result
RQ1	24/24 separate-implementation oracle cases (16 rows); regenerated
RQ2	10 rewrite families, 19 cases, 17 normal-form checks; 0 FactSet/charge mismatches; regenerated
RQ3	5 cases, each passing deterministically and as a PostgreSQL race test (5/5 and 5/5); Join/Union/Group paths; regenerated; campaign attack corpus (Section XX)
RQ4	30/30 gates; novel 2.791/2.909 s and replay 13.82/17.68 ms p50/p95; layout, contention, 1,035,013-Fact offline re-derivation (shared encoder); archived run at its recorded source hash
RQ5	Four 345,000-row epochs, 12 builds, max 77.055 s; 5/5 rollover checks across 3 transitions; regenerated
Campaign	`formal-v113-publication-05`: 33 fresh deployments of 11 profiles, 172 frozen cells, 9,393 samples, all passing; sealed at `faf4d095701b`

VIII. ORDINAL/BITMAP REFINEMENT

For each immutable dictionary segment j, the publication compiler checks a bijection $D_j : F_j \leftrightarrow [0, n_j)$ between canonical FactIDs and 32-bit ordinals. Let

$$\mathrm{Decode}_j(A) = \{D_j^{-1}(x) \mid x \in A\}.$$

The concrete OR and ANDNOT operators are ordinary exact set operations on canonical Roaring bitmaps, so decoding commutes with union and difference and popcount equals decoded cardinality. Dynamic release/outcome dictionaries obey the same invariant after exact hash/payload collision checks. Segments are disjointly tagged by dictionary and segment identity; cross-segment decoding is their disjoint union.

The streamed companion replaces each singleton Scan annotation with its publication ordinal. For one canonical output group the engine preserves the support bitmap and stores only multiplicities above one. Same-proof composition adds multiplicities, alternative-proof composition takes maximum, and a hash-ordered k-way merge emits the identical witness sequence used by the canonical commitment. Consequently decoding the three bitmap components recovers (F_r, F_i, F_o) exactly; no approximation or probabilistic membership test appears in the safety path.

The root state is one tuple (H_r, H_i, H_o, e) of three immutable set-manifest digests and an epoch. A transaction based on epoch e computes exact novelty with ANDNOT, checks all decoded cardinalities, and can publish the three candidate heads only together at epoch $e+1$. A stale candidate refreshes all three dimensions. This simulates the abstract transition $K'_d = K_d \cup F_d$ for every $d \in \{r, i, o\}$ and preserves the root-family theorem. The finite model `ExposureBitmapRefinement.tla` specifies this commuting relation and configured invariants for bounded dictionaries, requests, replay, and CAS conflicts; it is not a mechanized proof of the Go implementation and is not itself performance evidence. The accompanying action/transaction/test correspondence is an auditable refinement map, not a machine-checked refinement theorem.

The source-controlled ordinal/bitmap campaign measures 560 operations on 140 fresh roots in one fixed-host deployment. All 30/30 implemented report gates pass; at 12 Release, 1,035,000 dependency, and 1 Outcome facts, novel p50/p95 is 2.791/2.909 s and replay p50/p95 is 13.82/17.68 ms. This end-to-end report checks exact cardinalities and set/observation digests. A distinct, source-bound supplement now covers the obligations that report did not imply: 12 dense/clustered/random-sparse–overlap cells, public-path same-root batches of widths 1/4/8/16 through 2 enforcement-layer replicas, and an offline derivation-independent comparison of all 1,035,013 facts.

The layout result is a bitmap-kernel measurement, not FactGate end-to-end latency: 60 exact portable complete-set round-trip checks pass at their actual cardinalities, and the worst ANDNOT–popcount–OR p95 is 16.46 ms. The contention result uses a frozen ten-row publication with a one-row query effect, not a high-cardinality load test: exactly one contender per cell advances $B-1$ to B, all 25 aggregate losers add zero novelty, and all four $B+1$ attempts fail with released reservations and no partial content/result/observation. All 29 requests across the four cells enter transitive PostgreSQL lock-wait chains before blocker release. This demonstrates exercised contention and the subsequent single-epoch linearization, but does not identify the exact awaited statement and is not evidence of a stale read, CAS conflict, or retry count. A separate store-level regression deterministically queues two conditional head updates, observes the exported retry counter, and checks that all three dimensions are recomputed from the fresh head; it is test coverage, not a public-path measurement. The offline oracle reports 0 mismatches while checking every hash, payload, and witness multiplicity; it shares the canonical FactID specification and encoder and therefore supports a derivation-independent, not fully implementation-independent, claim.

An excluded diagnostic run found that the original failure path carried a success-only ordinal materialization into failure settlement. The budget check still withheld result and ledger changes, but the reservation remained pending and readiness degraded. The fixed path sanitizes that settlement, preserves the budget-exhaustion terminal code, and is covered by both a targeted regression and the final fresh concurrency campaign. The earlier 560-operation success-path report remains reproducible from its source snapshot; the final concurrency report is likewise bound to a fixed source identity. Source divergence is reported and is not attributed to either campaign.

TABLE VII
REPORTED NUMBER GROUPS, THEIR COMMITS, RETAINED FILES, AND REGENERATION COMMANDS.

Numbers	Commit	Retained file(s)	Regeneration
RQ1–RQ3 (Table X)	tracked at `4bc1652ec43f`	`evaluation/exposure/results.json`, `corpus.json` (digest `5704d22606b0`)	`make eval-exposure`
RQ4 archived ordinal/bitmap	recorded in `evidence/legacy-rq4-source.json` under the results directory	`evaluation/exposure-performance/results.json`, `path_analysis.json`	`make eval-exposure-performance`, `-scale`, `-storage`
RQ5 publication and routing	tracked at `4bc1652ec43f`	`evaluation/daily-publication/evidence/`, `daily-publication-online/evidence/`	`make eval-daily-publication`, `-online`
V5 outcome-set regressions and randomized root-family sequences	`4bc1652ec43f`	`evaluation/v5-outcome/evidence.json`, `raw/go-test.jsonl`, `compose-receipt.json`	`scripts/ record-compose-e2e.sh`
Publication campaign (main Table 1; Tables XV, XVI, XX, XXXVIII)	`faf4d095701b`	`evaluation/final-v5-wsl2/publication-evidence-v1.json` → `campaign-evidence.json`, `deployments/*/raw/*.jsonl`	`scripts/run-profile-campaign.sh`; macros: `make paper-evidence`
Attack corpus, adaptive trace, Scale, ProvSQL pairing, RQ5 cells (Sections XX–XXV)	`faf4d095701b`	the same campaign's `deployments/*/raw/*.jsonl` plus `evaluation/finalv5attack/corpus-v1.json` (digest `483b667369a0`)	`make paper-evidence`
Sub-phase pilot (Section XVII)	f8c9c53528fe, 9a0b2107d1ed	`evaluation/final-v5-wsl2/pilot-evidence-v1.json` → `raw/subphase-pilot-17`, `raw/subphase-pilot-13`	pilot; not publication evidence
ProvSQL base-tuple cross-check (Section XXIV)	9a0b2107d1ed	`evaluation/final-v5-wsl2/pilot-evidence-v1.json` → `raw/provsql-leaf-pilot-05`	pilot; not publication evidence
Retained pilots (Table VIII; counter arms, benign trace, footprint ladder, scale ladder, DP comparison, adversary: Sections XXIII-A, XXIII-C, XXIII-D, XXIII-F–XXIII-H)	per run, in the registry (Table VIII: `8aab6d4e4cac`)	`evaluation/final-v5-wsl2/pilot-evidence-v1.json` → pilot run directories	pilot; not publication evidence
Generated-plan campaign (Section XXVIII)	tracked at `4bc1652ec43f`	`evaluation/generatedalgebra/results.json` (digest `79e25395cb4b`)	`make eval-generated-algebra`
TPC-H lowerability (Section XXIX)	tracked at `4bc1652ec43f`	`evaluation/tpchlowerability/results.json` (digest `8320ec5f7131`)	`make eval-tpch-lowerability`
SSB lowerability (Section XXX)	tracked at `4bc1652ec43f`	`evaluation/ssblowerability/results.json` (digest `df69d2cc8692`)	`make eval-ssb-lowerability`
Agent-written workload (Section XXXI)	tracked at `4bc1652ec43f`	`evaluation/agentworkload/results.json` (digest `0621e3c3c137`)	`make eval-agent-workload-lowerability`
TLC (Table VI)	tracked at `4bc1652ec43f`	`formal/results/*.json`, `*.log`	`make formal`
Held-out workload and its corrected-sheet rerun (Section XXXI-A)	questions frozen at `cf49c49`	`evaluation/agentworkload-heldout/results.json` (digest `34b3432dcc5d`); `evaluation/agentworkload-heldout-sheet/summary.json` (digest `f23dca48ec74`)	static lowering, not executed; rerun: pilot, `run.sh` there
LLM-agent pilot (Section XXXI-B)	per run, in the registry	`evaluation/agent-pilot/results.json` (digest `0fedf0865a34`, run `p10-b2-agent-01`); `p10-pilot-evidence-v1.json` → raw	pilot; `evaluation/agent-pilot/analyze.py`
Throughput and client-retry pilots (Section XIX-A)	per run, in the registry	`evaluation/throughput-pilot/results.json` (digest `09f7b5221685`, run `p10-b6-throughput-02`), `throughput-retry/results.json` (digest `f186efa6fd5f`, run `p10-r2-c4b-throughput-retry-01`)	pilot
Cost ablation and naive ledger (Section XXIII-J)	per run, in the results file	`evaluation/b5-ablation/results.json` (digest `19a7e8a6f810`), `naive-scale-replay.json` (digest `84428949ffc3`)	pilot; arms i, iv executed, ii composed; naive ledger on the benign trace and, same-input, on the sealed Scale sets
Budget sweep (Section XXIII-I)	tracked	`evaluation/budget-utility-sweep/results.json` (digest `2aad00e07307`)	simulation, not executed; `evaluation/cmd/budget-utility-sweep`
Timing channel (Section XXIII-E)	per run, in the registry	`evaluation/timing-channel/results.json` (digest `fa98548b6422`; runs `pilot-counter-rigor-03`, `pilot-adversary-05`, `pilot-footprint-08`)	analysis of retained pilot records; `evaluation/timing-channel/analyze.py`
Aligned counter replay (Section XXIII-B)	tracked	`evaluation/counter-aligned/results.json` (digest `66c7ebf488bb`)	replay over the sealed trace; `evaluation/counter-aligned/replay.py`
Passant replay (Section XXIII-K)	Passant `4287864`	`evaluation/passant-comparison/results.json` (digest `354a51f14c8e`)	standalone replay; `evaluation/passant-comparison/replay.py`

A. Implementation Coverage

Table VIII distinguishes formal semantics, oracle coverage, online lowering, and PostgreSQL end-to-end exposure checks. "Online" is the public structured-plan path, not arbitrary SQL acceptance.

The online-lowering column states the accepted capability, whereas the PG E2E column states the measured coverage. The latter's two-source graph is a representative exercise of the binary join/effect step generated by the fold, not a limit on admitted graph width or topology; the sealed ProvSQL nonce-join cells additionally drive a three-source equijoin (orders, lineitem, nonce)

TABLE VIII
OPERATOR COVERAGE. PG E2E EXECUTES THE PAIRED STATEMENTS AND DERIVES THE EXPOSURE OBSERVATION RATHER THAN CHECKING RESULT EQUALITY ALONE.

Operator	Formal semantics	Independent oracle	Online lowering	PG E2E exposure check
Scan/Select/Project	yes	yes	one Catalog product	paired output/evidence and hidden keys
Group + aggregates + HAVING	yes	yes	above either online input	hidden group key and aggregate membership
Inner equijoin	yes	yes	connected graph of 2–16 distinct Catalog roles; one or more typed equalities per edge; canonical fold	representative two-source operand exchange, fanout, downstream SUM
Union-distinct	yes	yes	two filtered branches of one product; explicit full distinct schema	all members, hidden field, exchange, downstream COUNT
Page	yes	yes	one-product plan with stable Catalog suffix	offset/tie-break and withholding
Semantic View DAG	only after expansion to a typed core term	inherited from that expanded term	one-root composition into `CompileRelational`; depth/source 16; multi-equality INNER equijoin; one aggregate barrier	three-level discovery, drift, and transaction regressions; no separate View-DAG performance campaign

through the same end-to-end pipeline at all three measured scales. We do not report a source-count or topology performance campaign.

Separate source-level tests, outside the digest-bound PG campaign, exercise five-source left-/right-deep AST equivalence, a ten-source chain with two equalities per edge, star and cyclic graphs, shuffled node/edge/predicate order and direction, acceptance at the 16-source boundary, and rejection at 17 sources. They are implementation tests, not multi-source PG E2E evidence, performance measurements, or a mechanized proof.

B. Receipt Persistence-Time Semantics

The normal production enforcement path creates a *co-committed receipt*: receipt signing and insertion occur in the same Control-PG transaction as resource and exposure settlement, terminal and artifact-registration audit evidence, and the PENDING artifact row. The implementation also retains a compatibility recovery path for a terminal query, artifact row, and registration event that already exist while the receipt row is absent. This path first validates the original registration event against the immutable audit chain and then emits a *recovered attestation*. Such an attestation authenticates the same PENDING intent, but its `signed_at` can postdate the original settlement and the receipt itself was not co-committed with that settlement. Both forms prove intent rather than availability; the normal Compose path produces the co-committed form. They share one wire format and verification rule rather than carrying a recovery-mode flag.

IX. WORKED POLICY

The campaign's bounded trace profile binds every exposure dimension before any counter: for "review this department's receipts" it grants $B_R/B_D/B_O = 7/12/18$ next to a request budget of 110 and a row budget of 1,000, so on the 100-query trace the counters admit everything (250 rows in 100 queries) while the Dependency budget refuses at query 37, the first equivalent predicate that reads new base facts. The ladder shows B_O binding (5 atoms and composites refuse the third count probe); B_R binds in the sweep case: under the default low route ($B_R = 2{,}000{,}000$, $B_O = B_Q = 128$) paging a $10^5 \times 16$ Product in 100 pages releases 1.6×10^6 Facts and 100 composites, each page legal under any row limit, a repeat of the same pages is a zero-novelty replay, and a sweep of a second such Product is refused at B_R (budget arithmetic, not a measured cell); B_D sized to rows times aggregable fields bounds the *distinct* base facts any aggregate sequence touches, not how many run (the main text's threat model bounds settled and refused predicates). Ranking a hidden field is not admitted: an `ORDER BY` column that is not selected is rejected (`ORDER_FIELD_NOT_SELECTED`), so `SELECT id ... ORDER BY salary` must release `salary`.

X. PROFILE CALIBRATION BOUNDARY

The three budgets are human-governed declarations of distinct FactID capacity. The calibration-boundary table now lives in the main text (its Budget Calibration section); unit weights enable exact set union, not harm or differential-privacy accounting.

XI. Control Boundaries of the Closest Systems

Table IX lists, for the cited comparators, the controlled object and decision, the horizon and persistent subject, and the enforcement/release coupling; it is a boundary table drawn from the cited artifacts' modeled or paper-reported scopes, not a feature score; the works are cited in the main text.

TABLE IX
Representative control boundaries from the cited artifacts (modeled or paper-reported scopes, not a feature score).

System/family	Controlled object and decision	Horizon and persistent subject	Enforcement/release coupling
FactGate	Declared Result, Dependency, and Outcome FactID set novelty: does the next release fit the budget?	One signed, publication-bound root family; descendants share its persistent ledger	One three-dimensional root-head CAS co-commits ledger, audit, receipt, and PENDING intent; availability is recoverable
DFC/Passant	Aggregate policy predicate over a bag of provenance monomials: is this flow compliant?	One SQL execution; relational context may include identity, delegation, or system state; trajectory policy is future work	REMOVE/transform/KILL; source, required-source, and sink write policies
GAAP	Permission-consistent disclosure of IFC-labeled private data	Persistent state across execution steps and tasks separated in time	Deterministic disclosure confinement at agent/tool sinks
GProM/ProvSQL	General query provenance representation, rewriting, and circuits	Query-result derivations; no root-family budget	Capture and evaluate provenance; no cumulative release decision
Data Product MCP	Purpose-versus-contract compliance and pre-execution contract guards: may this query run for this purpose?	Per query; an audit chain links question, purpose, policy, SQL, and result	Reject or run before execution; no settlement or availability protocol
SafeNLIDB	Learned safety decision and SQL generation from schema, constraints, and interaction history	Multi-turn history supplied to the model; no persisted ledger	Reject or generate SQL at the model/NLIDB layer

XII. Clearance-Accounting Obligations and Evidence

The main text's accounting obligations O1–O4 read as follows.

- **O1 Output coverage.** Every selected cell $(t, a) \in R \times V$ of a successful result satisfies $\mathrm{mat}(R, t, a) \in \mathcal{F}_R(R, V)$ (the main text's observation equation); normalization may merge equal identities, so coverage is not row count.
- **O2 Positive dependency coverage.** $\mathcal{F}_D^{\varpi}$ is exactly the declared inductive footprint (retained predicate fields, join-key annotations, group keys, complete aggregate arguments), excluding rejected rows and non-output pages.
- **O3 Conservation.** The listed normal-form rewrites, projection splits, complete page partitions, idempotent overlaps/retries, join support, and root-ledger delegation cannot reduce cumulative charge; arbitrary SQL equivalence is not covered.
- **O4 Probe capacity.** Each success creates one composite and canonical predicate atoms; exact replay reuses them, while a changed canonical predicate component or normalized query/result observation changes the corresponding identity except for collision. Zero/empty results remain outcomes; a rejected attempt creates none and is charged only against the request budget. Cardinality is governance capacity, not entropy; one atom is not one bit of privacy loss.

Fix an approved root $T = (P, S, B, C)$ whose contract binds versioned profile ϖ. Let $K_X(T)$ denote the decoded Fact set in the corresponding committed root-family head—equivalently, its accounted set. We write $\mathcal{F}_X^{\varpi}(q)$ for the Result, Dependency, or Outcome component already defined by the main-paper effect under S and C, and $\Delta_T^X(q) = \mathcal{F}_X^{\varpi}(q) \setminus K_X(T)$. This notation exposes the profile parameter; it does not add a second exposure algebra. A selected visible cell is exactly a pair (t, a) with t in the successful result bag and a in its requested visible field set.

O1—Output coverage. Every selected visible cell contributes its validated materialized Result identity. Equal semantic identities may collapse under set normalization, so coverage is neither returned-row count nor one distinct FactID per wire-cell occurrence. Changing the selected field set changes the covered cells.

O2—Positive dependency coverage. The Dependency set is exactly the support selected by E-Scan, E-Project, E-Select, E-Page, E-Join, E-Union, and E-Group: retained predicate fields, join keys, group keys, distinctness fields, and complete aggregate arguments are included. It is not minimal causal provenance or a physical-read trace, and it omits rejected rows and non-output pages under this profile.

O3—Conservation. Only the listed typed normal-form rewrites, disjoint projection splits, complete page partitions, overlaps/retries, canonical join folds, and root-family delegation are covered. Their Fact-set union cannot be reduced by changing the admitted representation; arbitrary SQL equivalence is not claimed.

O4—Probe capacity. A successful query contributes canonical predicate atoms and one normalized query/result composite, even for zero or empty output. Exact replay reuses both; a changed canonical predicate or normalized query/result observation separates the corresponding identity except for collision. Cardinality is governance capacity, not entropy or privacy bits.

O5—No unaccounted governed release. If a canonical artifact for q is `AVAILABLE`, then each $\mathcal{F}_X^{\varpi}(q)$ must already be included in $K_X(T)$ and every $|K_X(T)| \le B_X$. Settlement can precede availability, so the reverse implication is false. Staging, failed settlement, and a bare settlement receipt do not cross this boundary.

O6—Profile binding. Query derivation, settlement, semantic replay, receipt verification, and audit must agree on the task, Grant, Catalog, publication/dictionary, profile, normal form, View contract, type/collation, and result handling. The receipt

directly records the profile version and observation/component digests; the complete environment is transitively bound, not represented by a `profile_digest` field.

Table X is the claim-evidence matrix. "TLA+" means a bounded invariant in the stated finite configuration. A blank semantic extraction in TLA+ is not repaired by calling an implementation test a proof.

TABLE X
CLEARANCE SOUNDNESS OBLIGATIONS AND ASSURANCE LAYERS.

Claim	Paper semantics/theorem	Bounded TLA+ or transaction	Executable evidence
O1	Result extraction and representation closure	TLA+ accepts abstract candidate sets; it does not derive visible cells	Independent RQ1 oracle; ordinal-versus-algebra derivation tests
O2	Complete operator rules; dependency determinism; group/distinct/aggregate completeness proofs	TLA+ copies abstract Dependency candidates without modeling the rules	RQ1 memberships and hidden predicate/join/group/aggregate PostgreSQL checks
O3	Typed-normal-form, base-ledger, Outcome replay, and root-family theorems	`ExposureLedger` set novelty/family invariants; bitmap CAS refresh; no SQL rewrite model	RQ2 rewrite campaign; RQ3 split/page/retry/delegation/concurrency tests
O4	Atom/composite definition and Outcome separation theorem	Outcome shape and exact-set invariants only; no predicate payload or entropy model	zero/empty predicate separation, replay, full-hash and radix tests
O5	Settlement definition and the one-way release Property	Separate `ExposureLedger` budget and `ArtifactPublication` availability/hash invariants; Control settlement and AVAILABLE transactions link them	over-budget withholding, promotion failure/recovery, V8 intent, availability-inclusion, final-campaign receipt/object composition, and fault-injection tests
O6	Approved-task/profile definition and replay descriptor	No profile/Catalog/publication variables in these TLA+ models	replay-key binding, foreign-Catalog dictionary, View drift, receipt-signature, and publication-separation regressions

Table XI records the finite configuration and the TLC run behind each bounded model cited above; the checked configuration digests and complete logs are retained under `formal/results/`.

TABLE XI
BOUNDED TLC CONFIGURATIONS (TLC 1.7.1, ALL INVARIANTS PASSED, CHECKED 2026-08-09). BOUNDS ARE THE `.CFG` CONSTANTS; STATES ARE GENERATED/DISTINCT WITH THE SEARCH DEPTH.

Model	Finite bounds	Checked invariants	States (depth)
`ExposureLedger`	one root and one child task; 2 requests; 2 Release, 2 Dependency, 2 Outcome facts (1 atom, 1 composite); limits 1/1/2; 1 replay	TypeOK, TripleBudgetSafety, AccountedIffSettled, ExactNovelCharge, NoChargeWithoutSettlement, RejectedResultsStayBuffered, DerivedEvidenceMatchesBuffer, NovelChargesDoNotOverlap, TaskFamilyNonAmplification, SettledQueriesExecutedOnce	410,766 / 148,706 (12)
`ExposureBitmap Refinement`	2 requests; 3 dimensions; 2 facts and 2 ordinals over a base and a dynamic segment; 1 fact per request; 1 replay	DictionaryBijection, TypeOK, DecodeOrRefinement, DecodeAndNotRefinement, PopcountRefinement, DerivationRefinement, LedgerRefinement, ExactNovelCharge, TripleBudgetSafety	122,976 / 60,680 (10)
`OutcomeSet AbstractRefinement`	3 hashes; candidates $\{f_1, f_2\}$ and $\{f_2, f_3\}$; Outcome limit 3	TypeOK, ExactDifference, NoDoubleCharge, BudgetSafety	20 / 10 (4)
`ArtifactPublication`	2 requests; one expected canonical hash; 2 promotion retries	TypeOK, AvailableImpliesSettledCanonicalHashMatch, AbsentCanonicalHasZeroHash, RejectedNotAvailable, RetryRequiredStaysPending, RecoveryHasZeroLedgerDelta	1,324 / 441 (17)

No single row asserts a formally verified implementation. In particular, `ArtifactPublication.tla` checks only that its abstract AVAILABLE state implies its local settled flag and a matching canonical hash; the separate ledger models check novelty and bounds. The implementation correspondence is the Control-PG sequence that co-commits root observation/effect settlement, terminal and registration audits, a `PENDING` artifact intent, and the V8 receipt, followed by canonical-object verification and a second transaction that co-commits `AVAILABLE` with `QUERY_RESULT_CONSUMED`. The source-controlled final-campaign adapter uses one fail-closed verifier to compare the signed Grant/Catalog/profile and effect fields with an independent Control projection, prove the terminal, registration, and availability events on one audit path, re-hash the raw canonical ciphertext, and check the delivered Parquet size, identity, and normalized schema. This is combined executable evidence, not a public audit SDK, mechanized cross-module refinement, or liveness guarantee.

The receipt uses `observation_sha256` together with the component digests `release_set_sha256`, `influence_set_sha256`, and `outcome_set_sha256` (the wire and macro name `influence` denotes the paper's Dependency dimension throughout); it has no field named `candidate_digest`, `effect_digest`, or `profile_digest`. Hash, binding, or audit mismatch fails closed. Residual quantities and channels remain outside the manifest: differential privacy, total/background knowledge, timing, uncharged refusal, information content of rich predicates, fresh-root capacity, and cross-publication durable knowledge or deduplication.

TABLE XII
THREE-WAY RECONCILIATION.

Layer	Bound evidence	Required reconciliation	Insufficient alone
Effect manifest	Three candidate sets, `observation_sha256`, component digests, profile and publication context	Same query observation must be the effect settled under the bound root	Does not prove ledger commit or availability
Root-family ledger	Three committed heads, exact novelty/cardinalities, root epoch, settlement and terminal audit	Head transition and query observation must identify the manifest and remain within all bounds	Does not prove object promotion
Artifact and receipt	Parquet/object hashes, registration/PENDING intent, V8 receipt, canonical verification, availability audit	One final-campaign verifier reconciles the same query, signed intent, Control projection, canonical bytes, Parquet schema, and AVAILABLE event	Receipt alone proves PENDING intent; availability event cannot replace settlement

TABLE XIII
CHARGES FOR ONE AUDITABLE FIXTURE, NOT PERFORMANCE RESULTS.

Baseline	Charge
Query count	1
Returned rows	5
Serialized bytes	79
Weighted cells	15
Provenance without history	36
Full, first execution (r, i)	(15,36)
Full, replay (r, i)	(0,0)

XIII. DESCRIPTIVE BASELINE CONTRAST

For one grouped sum/count fixture, Table XIII reports six descriptive accounting baselines. Their units differ, so the table is not a ranking. It demonstrates why row count alone misses dependency facts and why history is required for zero incremental replay.

XIV. RETAINED BASELINE PILOT: PER-CELL GOVERNED COST

Table XIV expands the main text's summary of the retained Baseline run. It is a pilot-class run (`campaign_class pilot`, `publication_eligible false`): one deployment, 1 samples per cell after warmups, all 55 samples passing with a verified Receipt on every governed arm. It is not a completed publication campaign and must not be described as one.

Two properties are visible per cell. First, governed cost tracks the Dependency set rather than the released answer: S2 returns 3 grouped rows at either scale factor, yet its SF10 query depends on 1,200,000 Facts and costs more than cells returning tens of thousands of rows. A dash marks a mode the cell's contract does not declare: S5 has no normalized rewrite and S6 declares no replays at all. Second, replay latency is near-constant in result size, so the ratio between a governed replay and direct re-execution improves as the underlying query grows. At the scales measured here that ratio favors replay only once: the idempotent replay at S2/SF10 (22.88 ms) undercuts direct re-execution (24.93 ms) while issuing no business SQL; every semantic replay median in the table exceeds its cell's direct median.

XV. PUBLICATION CAMPAIGN: PER-CELL GOVERNED COST

Table XV expands the main text's summary of the frozen publication campaign `formal-v113-publication-05` (`campaign_class publication`, `publication_eligible true`): 33 fresh deployments of the submission commit `faf4d095701b`, 90 measured samples per Baseline cell across three independent executions, all 5,220 samples passing with a verified Receipt on every governed arm. The retained pilot of Section XIV is the earlier single-deployment run of the same cells and is kept only as mechanism evidence.

Charging separates cleanly from latency. Across every replay sample the business database was not contacted, and the semantic and normalized-rewrite modes settled all three Fact budgets at zero while recording the same actual Result, Dependency, and Outcome sets as the novel query they replay. Latency does not follow: in every cell of Table XV the semantic replay median exceeds the direct median, and only the idempotent replay at S2/SF10 undercuts direct re-execution. Replay saves business-database work and Fact charge, not client-visible time, at these scales. Idempotent replay instead returns the original terminal record, and its no-recharge property is carried by the control-plane snapshot being byte-identical before and after. One qualification belongs with these numbers: the per-Task row budget is a cumulative ledger and is charged on replay, so replay is free of Fact charges but not of row budget.

TABLE XIV
RETAINED BASELINE PILOT, PER CELL, IN MILLISECONDS (1 MEASURED SAMPLE PER CELL AFTER WARMUPS). OVERHEAD IS NOVEL OVER DIRECT. NOT PUBLICATION EVIDENCE.

Cell	Rows	Release	Dependency	Direct	Novel	Semantic	Idempotent	Overhead
S1/SF1	5,000	10,000	20,000	2.65	174.05	81.97	25.89	65.7×
S1/SF10	50,000	100,000	200,000	12.34	769.80	121.47	44.38	62.4×
S2/SF1	3	9	120,000	11.67	483.96	83.07	21.83	41.5×
S2/SF10	3	9	1,200,000	24.93	3272.63	78.80	22.88	131.2×
S3/10k-50k	10,000	10,000	50,000	2.50	237.58	83.38	25.15	95.1×
S3/1k-5k	1,000	1,000	5,000	0.61	121.74	76.88	22.07	198.1×
S3/45k-225k	45,000	45,000	225,000	7.25	580.45	105.71	39.29	80.0×
S5/SF1	5,000	10,000	20,000	4.48	308.86	91.07	25.15	69.0×
S5/SF10	50,000	100,000	200,000	17.59	1744.28	128.24	42.76	99.2×
S6/100k-x16	100,000	1,600,000	1,700,000	137.30	16976.27	–	–	123.6×
S6/100k-x4	100,000	400,000	500,000	38.23	4033.32	–	–	105.5×
S6/100x16	100	1,600	1,700	0.67	137.07	–	–	206.1×
S6/100x4	100	400	500	0.48	100.32	–	–	208.8×
S6/10k-x16	10,000	160,000	170,000	13.48	1737.33	–	–	128.8×
S6/10k-x4	10,000	40,000	50,000	4.98	377.61	–	–	75.8×

TABLE XV
PUBLICATION CAMPAIGN BASELINE, PER CELL. MEDIANS IN MILLISECONDS OVER 90 SAMPLES (THREE FRESH EXECUTIONS). OVERHEAD IS NOVEL OVER DIRECT.

Cell	Rows	Release	Dependency	Direct	Novel	Semantic	Idempotent	Overhead
S1/SF1	5,000	10,000	20,000	2.91	100.51	61.80	27.44	34.5×
S1/SF10	50,000	100,000	200,000	12.62	326.93	108.95	49.76	25.9×
S2/SF1	3	9	120,000	14.14	384.32	62.28	24.37	27.2×
S2/SF10	3	9	1,200,000	30.42	2459.32	62.65	24.12	80.8×
S3/10k-50k	10,000	10,000	50,000	2.43	151.06	62.34	28.65	62.1×
S3/1k-5k	1,000	1,000	5,000	0.83	97.15	56.61	25.01	116.9×
S3/45k-225k	45,000	45,000	225,000	8.67	319.42	88.91	43.67	36.9×
S4/depth-4	3	9	120,000	13.89	429.24	72.95	–	30.9×
S5/SF1	5,000	10,000	20,000	4.61	224.39	65.60	26.20	48.7×
S5/SF10	50,000	100,000	200,000	19.52	1431.39	112.37	50.19	73.3×
S6/100k-x16	100,000	1,600,000	1,700,000	173.89	3797.28	–	–	21.8×
S6/100k-x4	100,000	400,000	500,000	51.14	1133.42	–	–	22.2×
S6/100x16	100	1,600	1,700	0.90	129.82	–	–	144.0×
S6/100x4	100	400	500	0.76	85.20	–	–	112.0×
S6/10k-x16	10,000	160,000	170,000	16.28	586.97	–	–	36.0×
S6/10k-x4	10,000	40,000	50,000	5.33	201.72	–	–	37.8×

XVI. PUBLICATION CAMPAIGN: GATEWAY PHASE BREAKDOWN

The Gateway reports six non-overlapping server phases per governed response (prepare, execute-and-derive, artifact stage, Control settlement, artifact publication, response finalize), and each sample's phases sum to its server total. Table XVI gives per-phase medians of the novel arm for the cells the main text discusses, over 90 samples each; per-phase medians need not add up to the median total. The dominant phase of S1/SF1 is `execute_and_derive` (32.3% of its server total), of S2/SF10 `execute_and_derive` (93.6%), and of S6/100k-x16 `execute_and_derive` (77.2%); the Control settlement transaction takes 20.59, 108.21, and 296.95 ms in those cells.

XVII. SUB-PHASE PILOT: INSIDE EXECUTE-AND-DERIVE

The Gateway reports finer component timers than the six pipeline phases (`component_ms`), but the publication campaign's adapter did not retain them. Pilot-class amendments re-ran the `analytics-orders`, `analytics-orders-lineitem`, and `result-heavy` profiles with the adapter retaining the component map: one deployment each of the first two in run `subphase-pilot-17` (one untimed warmup request per cell before its sample, so that no cell's only sample is the deployment's first request; an earlier run without warmups measured a cold first request for S1/SF1 and is retained but superseded) and three deployments of `result-heavy` in run `subphase-pilot-13`, so 1 novel sample per cell, all pilot class (`campaign_class pilot`, `publication_eligible false`); the table reports medians over the pooled samples. Table XVII lists the atomic components of the novel arm for the three cells the main text discusses. Two Gateway timers are composites and are not tabulated: `ordinal_stream` equals the companion SQL plus the stream consumer, and `bitmap_derivation` equals visible preparation plus consumer plus finish (`internal/gateway/query.go`).

TABLE XVI
GATEWAY PHASE MEDIANS OF THE NOVEL GOVERNED QUERY (MS) FROM THE PUBLICATION CAMPAIGN. EACH SAMPLE'S SIX PHASES ARE NON-OVERLAPPING AND SUM TO ITS SERVER TOTAL; THE COLUMNS ARE PER-PHASE MEDIANS OVER 90 SAMPLES, SO THEY NEED NOT ADD UP TO THE MEDIAN TOTAL.

Cell	Prepare	Exec.+derive	Stage	Settle	Publish	Finalize	Total
S1/SF1	11.23	26.02	6.97	20.59	14.79	0.49	80.49
S2/SF10	17.63	2285.93	7.38	108.21	14.44	0.49	2442.51
S6/100k-x16	16.77	2631.35	398.06	296.95	61.07	0.54	3409.20

TABLE XVII
SUB-PHASE PILOT: GATEWAY COMPONENT TIMINGS (MS, MEDIANS OVER 1 NOVEL SAMPLE PER CELL) FOR S1/SF1, S2/SF10, AND S6/100K-X16.

Component	S1/SF1	S2/SF10	S6/100k-x16
visible SQL (PostgreSQL)	2.5	28.7	149.2
companion SQL (PostgreSQL)	6.7	548.1	178.9
visible preparation	0.0	0.1	0.1
ordinal stream consumer (Go)	9.6	1611.4	941.9
witness merge and finish (Go)	5.1	237.5	1183.5
result encoding (Parquet)	6.8	15.7	374.8
settlement persist	19.7	123.0	284.6
execute-and-derive (pipeline phase)	25.8	2429.9	2463.8
server total	82.9	2605.7	3203.1

Settlement persist belongs to the Control settlement phase and is listed for scale. The split shows that exactness is paid in Go-side ordinal consumption and witness merging, not in PostgreSQL: the companion statement is 576.8 ms of a 2429.9 ms phase at S2/SF10 and 328.1 ms of 2463.8 ms at S6/100k-x16. These are pilot measurements on one host and are cited only for the breakdown, not for latency.

XVIII. PUBLICATION CAMPAIGN: BETWEEN-EXECUTION DISPERSION

Each profile was deployed three times from scratch and every Baseline cell was measured in each deployment. Table XVIII reports, per cell, the median of the novel and direct arms within each execution and the spread of those three medians relative to the pooled median; the concurrency rounds are reported the same way in Table XIX. The executions ran sequentially on the same host without cache resets, so the spread bounds run-to-run variation on one warm host, not cross-host variation.

XIX. PUBLICATION CAMPAIGN: SAME-ROOT CONCURRENCY

The concurrency profile of the same campaign measures one root family under rounds of concurrent governed requests at its budget boundary: the root has one unit of capacity left, so exactly one contender per round can settle novel work while the others settle zero novelty or are refused. One sample is one round, and its drain time runs from launching all contenders until every one has returned. In `forced_queue_safety` a blocker transaction holds the root-head row lock until every contender has arrived and is waiting on it, then releases the whole round at once; in `natural_contention` the contenders run freely and the round counts only if the production Outcome set reported at least one CAS conflict. Table XX reports round drain time per cell over 450 rounds; requests per second is the width over the median drain. The conditioning discarded nothing: the sealed sample files retain every generated round, 30 passing rounds per cell in each of the three deployments and no invalid round. All rounds ran on one host against one enforcement instance.

A. Successful throughput under an ample budget (pilot)

The cells above sit at a budget boundary by design, so one contender per round settles novel work. A reviewer asked the complementary question: with budget to spare, how many legitimate novel queries does the Gateway keep completing per second on one root head and across independent roots, at what tail latency, and with how many conflicts? The pilot `p10-b6-throughput-02` (design frozen before execution in `docs/p10_b6_throughput_experiment_design.md`; results digest `09f7b5221685`) answers it on three fresh deployments of the benign-x4 profile, whose approval route grants 896 queries, 100,000 rows and 107,628 / 5,800,000 / 224 Release / Dependency / Outcome facts per root, over the 100,000-row `final_v5_result_heavy` Product. A cell provisions K root tasks with N delegated contenders each through the real request/OA/grant path; in each of two rounds every contender issues one `query_sql` at once and the round drains when the last returns. Footprints are *disjoint* (one distinct row per contender), *nested* (`row_id <= 10i`), or *identical* (the same row for all), offset between rounds so no round repeats a footprint. Table XXI reports 72 rounds. Two properties are reported

TABLE XVIII
PER-EXECUTION MEDIANS (MS) OF THE NOVEL AND DIRECT ARMS FOR EVERY BASELINE CELL, WITH THE RELATIVE SPREAD OF THE THREE MEDIANS.

	Novel				Direct			
Cell	Exec. 1	Exec. 2	Exec. 3	Spread	Exec. 1	Exec. 2	Exec. 3	Spread
S1/SF1	99.25	101.14	99.84	1.9%	2.92	2.96	2.83	4.2%
S1/SF10	326.64	331.05	324.03	2.1%	12.95	12.87	11.97	7.7%
S2/SF1	381.47	389.87	382.97	2.2%	14.33	14.12	14.03	2.1%
S2/SF10	2471.46	2457.84	2448.67	0.9%	30.44	30.85	29.93	3.0%
S3/10k-50k	150.71	150.43	152.11	1.1%	2.50	2.41	2.43	3.4%
S3/1k-5k	97.19	95.89	97.96	2.1%	0.83	0.85	0.81	4.6%
S3/45k-225k	323.71	316.91	319.26	2.1%	8.67	8.71	8.63	1.0%
S4/depth-4	430.86	430.62	424.44	1.5%	13.79	13.73	14.18	3.3%
S5/SF1	223.18	227.02	225.65	1.7%	4.64	4.76	4.49	5.7%
S5/SF10	1425.78	1440.68	1428.45	1.0%	19.73	20.27	18.74	7.9%
S6/100k-x16	3841.42	3802.29	3736.70	2.8%	175.12	173.94	172.28	1.6%
S6/100k-x4	1154.33	1096.85	1134.16	5.1%	52.87	51.34	50.47	4.7%
S6/100x16	127.23	130.26	129.97	2.3%	0.96	0.92	0.86	10.5%
S6/100x4	83.42	87.62	84.00	4.9%	0.77	0.74	0.76	4.4%
S6/10k-x16	593.29	595.05	574.01	3.6%	16.42	16.56	15.89	4.2%
S6/10k-x4	200.29	201.67	202.67	1.2%	5.24	5.53	5.26	5.5%

TABLE XIX
PER-EXECUTION MEDIAN ROUND DRAIN (MS) OF EVERY CONCURRENCY CELL, WITH THE RELATIVE SPREAD OF THE THREE MEDIANS.

Width	Mode	Exec. 1	Exec. 2	Exec. 3	Spread
1	serial	134.1	135.4	131.6	2.8%
10	forced_queue_safety	651.2	654.0	608.7	7.0%
10	natural_contention	657.4	636.5	623.8	5.3%
50	forced_queue_safety	4745.8	4634.8	4482.0	5.7%
50	natural_contention	4732.8	4700.1	4491.0	5.2%

separately because they are different things: ledger consistency (no round errored in the harness, and in every round each root's ledger moved by exactly the sum of the responses' charges) and request completion (a request that the head CAS gives up on returns `CONFLICT`, uncharged, and is not settled; the table counts these per cell, and the first reading below gives the fraction).

Three readings. First, one root head settles about 28–29 novel queries per second whether ten or fifty contend, because every novel settlement serializes on the head: fifty contenders mostly wait, the p95 grows from 344 ms to 1446 ms, and the Outcome-set CAS records 1428 conflicts in 1689 attempts. At fifty on one head the retry budget is not always enough: the settlement retries the head CAS up to sixteen times (`ordinalCASMaxAttempts` in `internal/control/result.go`) and then returns the public `CONFLICT` code; 76 of the 600 requests in the one-root, fifty-contender cells ended that way (five to eight per round), 0 elsewhere, and the caller's only recourse is to resubmit. Nothing was charged for them, so "no error round" and "every request settled" are not the same statement: in the one-root, fifty-contender cells about one request in eight did not complete within its round. The harness did not resubmit, so the completion rate and end-to-end latency a client would see with retries are not measured by this pilot; the follow-up at the end of this subsection measures them. Second, four independent roots raise novel throughput to 66 per second, about $2.3\times$ one root rather than $4\times$, with the p95 at 2886 ms for 200 simultaneous contenders: beyond the head, the single host's Gateway and database are the next bound. Third, identical footprints settle fastest (70 per second at one root, fifty contenders) because all but one per root are semantic replays that touch no business data and add no novelty, the replay behaviour the main text describes. No budget refusal occurred; the design's arithmetic (two rounds keep the largest cell under the Outcome floor) held. The numbers are one laptop-class host with everything colocated, and the pilot does not vary the retry limit or the settlement batching the main text names as the ways to relax the head bound.

a) With client resubmission.: A follow-up pilot (`p10-r2-c4b-throughput-retry-01`; design frozen before execution in `docs/p10_r2_c4b_throughput_retry_design.md`, results digest `f186efa6fd5f`) reruns the fifty-contender cells on three fresh deployments with a client that resubmits a `CONFLICT` refusal up to 3 times after a 50–150 ms jittered pause, each resubmission under a fresh `request_id` because the refused query is a settled `FAILED` record that an idempotent retry would replay. The retry count and the pause were fixed a priori (the server already retries the head sixteen times per attempt), not fitted. A zero-retry control pass on the same deployments reproduces the first pilot: 78 of 600 one-root requests refused. With resubmission, all 600 of 600 one-root disjoint and nested requests settled (100.0%): 89 resubmissions were issued, 89 requests settled on a resubmission and 0 remained refused; the four-root cells needed none (0 refused). The

TABLE XX
PUBLICATION CAMPAIGN CONCURRENCY CELLS: ROUND DRAIN TIME (MS, TYPE-7 P50/P95) BY WIDTH AND RELEASE MODE, AND SETTLED REQUESTS PER SECOND AT THE MEDIAN.

Width	Mode	Rounds	Drain p50	Drain p95	Req./s
1	serial	90	133.4	158.2	7.5
10	forced_queue_safety	90	647.6	786.6	15.4
10	natural_contention	90	633.9	779.6	15.8
50	forced_queue_safety	90	4606.2	5394.1	10.9
50	natural_contention	90	4642.0	5353.3	10.8

TABLE XXI
SUCCESSFUL-THROUGHPUT PILOT (THREE DEPLOYMENTS, TWO ROUNDS EACH): ROOTS K, CONTENDERS PER ROOT N, OVERLAP, ROUNDS, NOVEL/SETTLED/REQUESTED REQUESTS, REQUESTS REFUSED WITH `CONFLICT` (UNCHARGED, NOT SETTLED), SETTLED AND NOVEL REQUESTS PER SECOND (MEDIAN OVER ROUNDS), CLIENT P50 AND P95 LATENCY (MS, MEDIAN OVER ROUNDS, OVER THE REQUESTS THAT RETURNED), OUTCOME-SET CAS CONFLICTS/ATTEMPTS SUMMED OVER THE ROUNDS, AND WHETHER EVERY ROUND'S LEDGER MOVEMENT MATCHED THE CHARGES.

K	N	Overlap	Rounds	Novel/settled/req.	`CONFLICT`	Settled/s	Novel/s	p50	p95	Conflicts/attempts	Ledger
1	10	disjoint	6	60/60/60	0	28	28	243	344	240/300	yes
1	10	nested	6	60/60/60	0	22	22	321	435	243/303	yes
1	10	identical	6	6/60/60	0	47	5	203	210	28/34	yes
1	50	disjoint	6	261/261/300	39	29	29	1097	1446	1428/1689	yes
1	50	nested	6	263/263/300	37	25	25	1267	1706	1452/1715	yes
1	50	identical	6	6/300/300	0	70	1	641	707	15/21	yes
4	10	disjoint	6	240/240/240	0	59	59	546	661	322/562	yes
4	10	nested	6	240/240/240	0	56	56	583	676	261/501	yes
4	10	identical	6	24/240/240	0	65	6	554	602	13/37	yes
4	50	disjoint	6	1200/1200/1200	0	66	66	2445	2886	1847/3047	yes
4	50	nested	6	1200/1200/1200	0	60	60	2867	3215	1678/2878	yes
4	50	identical	6	24/1200/1200	0	79	2	2193	2462	9/33	yes

cost is latency, not throughput or correctness: the disjoint cell's end-to-end p95, backoff included, is 1656 ms against 1452 ms for the final attempt alone, novel settlement stays at 29 per second, and every round's ledger movement equals the charges (all), a refused attempt having charged nothing. Table XXII lists every cell.

XX. PUBLICATION CAMPAIGN: ATTACK CORPUS

The attack profile of campaign `formal-v113-publication-05` replays the frozen corpus `evaluation/finalv5attack/corpus-v1.json` (SHA-256 prefix `483b667369a0`, bound into every retained sample) against a fresh root task per sample: 15 cells—7 sequences, each in a direct PostgreSQL arm and a governed novel arm, the request-identity sequence also in both replay arms—with 9 samples per cell over three deployments, 135 samples in all. The generator requires the per-step outcome of every sample in a cell to be identical, the per-step charges to sum to the sample's settled charge, every decomposition to have charged exactly its complete query's Result and Dependency Facts, and the threshold ladder to end in exactly one `EXPOSURE_BUDGET_EXHAUSTED` refusal (`ROOT_OUTCOME_CEILING_EXCEEDED`, an evidence-side projection from the oracle; the client receives only the code) at its preregistered Outcome ceiling of 5. Table XXIII summarizes each governed sequence and Table XXIV traces the threshold ladder and Table XXV lists every step with its actual and charged Facts. The direct arm charges nothing and refuses nothing: it answered every step, including both set-operation variants and the third threshold probe. The corpus is preregistered; it does not search a strategy space, and its Outcome charges follow the frozen profile's page and range composites.

XXI. PUBLICATION CAMPAIGN: PROVSQL-PAIRED PROVENANCE CAPTURE

The ProvSQL profile of the same campaign pairs three real systems on the same host and byte-identical SQL (every FactGate sample's committed Dependency set is also linked member by member to the independent oracle's closed-form semantic set for that scale and nonce, as in the Scale profile: 270 verifications, up to 1,305,003 Facts, none with a missing or extra ordinal; in the sealed campaign ProvSQL's own circuit is drained only to its per-output-row roots, so the pairing there is a cost reference; Section XXIV expands the circuit to base tuples in a pilot-class rerun): PostgreSQL 16.14 from a digest-pinned image, ProvSQL 1.11.0 built on that exact base image, and FactGate's public `query_sql` path with settlement, receipt, and AVAILABLE delivery. The query is the orders–lineitem equijoin over the first 1,000, 10,000, or 45,000 orders of a deterministic 50,000-order, 250,000-line fixture (TPC-H-shaped, generated from repository formulas, not the official benchmark), grouped by order status into three rows carrying a price sum, a line-number sum, and a count, and joined to one provenance-tracked nonce row that changes on every execution so that ProvSQL builds a novel circuit each time. The timed interval on every arm is the

TABLE XXII
THROUGHPUT WITH CLIENT RESUBMISSION (THREE DEPLOYMENTS, TWO ROUNDS PER CELL AND PASS): PASS, ROOTS K, CONTENDERS N, OVERLAP, RETRY LIMIT, ROUNDS, SETTLED/REQUESTED, COMPLETION (%), REQUESTS SETTLED ONLY AFTER A RESUBMISSION, RESUBMISSIONS ISSUED, REQUESTS STILL REFUSED, NOVEL PER SECOND (MEDIAN), FINAL-ATTEMPT P95, END-TO-END P95 AND MAXIMUM INCLUDING BACKOFF (MS, MEDIAN OVER ROUNDS), LEDGER MATCH.

Pass	K	N	Overlap	k	Rnd	Settled/req.	%	After	Resub.	Still	Nov/s	p95	e2e p95	e2e max	Ledger
control	1	50	disjoint	0	6	262/300	87.3	0	0	0	27	1575	1575	1614	yes
control	1	50	nested	0	6	260/300	86.7	0	0	0	23	1845	1845	1884	yes
control	1	50	identical	0	6	300/300	100.0	0	0	0	1	747	747	762	yes
retry	1	50	disjoint	3	6	300/300	100.0	44	44	0	29	1452	1656	1741	yes
retry	1	50	nested	3	6	300/300	100.0	45	45	0	24	1722	2017	2081	yes
retry	1	50	identical	3	6	300/300	100.0	0	0	0	1	748	748	769	yes
retry	4	50	disjoint	3	6	1200/1200	100.0	0	0	0	64	2981	2981	3112	yes
retry	4	50	nested	3	6	1200/1200	100.0	0	0	0	58	3320	3320	3439	yes
retry	4	50	identical	3	6	1200/1200	100.0	0	0	0	2	2606	2607	2652	yes

TABLE XXIII
ATTACK CORPUS, GOVERNED ARM, PER SEQUENCE: STEPS, ACCEPTED STEPS, REFUSAL CODE, THE COMPLETE QUERY'S ACTUAL RESULT/DEPENDENCY FACTS WHERE THE SEQUENCE HAS ONE, AND THE FACTS CHARGED OVER THE WHOLE SEQUENCE.

Sequence	Steps	Acc.	Refused	Complete R/D	R	D	O
A-pagination/complete-to-pages	6	6	–	12/18	12	18	5
A-pagination/pages-to-complete	6	6	–	12/18	12	18	5
B-equivalent-sql/variants-v1	5	4	SQL_NOT_LOWERABLE	–	2	3	2
C-request-id/same-and-different	1	1	–	–	6	9	1
D-split-union/complete-to-split	4	3	SQL_NOT_LOWERABLE	12/18	12	18	6
D-split-union/split-to-complete	4	3	SQL_NOT_LOWERABLE	12/18	12	18	6
E-threshold/preregistered-v1	6	5	EXPOSURE_BUDGET_EXHAUSTED	–	13	18	5

client's complete drain of the ordered result; on ProvSQL it therefore includes query rewriting, aggregate-token construction, and persistent circuit construction, and excludes semiring evaluation, probability computation, and Shapley values; on FactGate it includes derivation, settlement, publication, and delivery. Warm business-data cache, disabled parallel gather, the simple text protocol, and a recorded per-statement timeout hold on both database sessions; execution order within each pair is randomized from a recorded seed. Table XXVI reports medians over 90 samples per cell (810 in all). The boundary is asymmetric by construction and the ratios price exact typed-set derivation plus settlement against general circuit capture; they do not rank the systems.

XXII. PUBLICATION CAMPAIGN: ADAPTIVE TRACE VERSUS ROW-LEVEL SECURITY

The RLS profile of campaign `formal-v113-publication-05` replays the frozen 100-query adaptive trace of `evaluation/finalv5rls/corpus-v1.json` (six receipt look-ups, 30 pages, 30 equivalent predicate spellings, 30 repeated aggregations, and four branches chosen from prior scalars) through three arms on the same `expense_detail` Product: PostgreSQL row-level security (a permissive department policy with `FORCE ROW LEVEL SECURITY` and column grants for a non-owner, non-superuser role), FactGate with an unlimited exposure ledger, and FactGate bounded at $\lfloor 0.7\,U \rfloor$ of the trace's distinct facts, where U is computed before any bounded run by the corpus package's trace-union oracle. That package deliberately imports no production Gateway, Control, exposure, Catalog, lowering, or query-plan code. Each arm ran 9 times over three deployments (54 samples with the policy-denied control), every sample of an arm recording the same per-step outcome; the generator requires the unlimited arm's final ledger to equal the oracle union and the bounded arm to refuse at the first prefix whose oracle cardinality exceeds a budget and to release nothing afterwards. Table XXVIII summarizes the arms; the main text's trace figure plots the oracle's prefix cardinalities. The policy-denied control targets a receipt outside the policy department together with a column outside the grant: both controls return zero rows and refuse at the column (RLS `42501`, FactGate `COLUMN_NOT_APPROVED`).

Counter arms are additionally executed end to end as a retained pilot (Section XXIII); within the sealed campaign itself, counter arms are derived from the same sealed samples by admission arithmetic, with budgets set by the same $\lfloor 0.7\,U \rfloor$ rule as the bounded arm and therefore calibrated on the reported trace (stated once here): each query's Release/Dependency/Outcome facts as counted by the independent oracle (`oracle_trace`) and its rows are replayed under a cumulative row budget and a query-count budget calibrated to refuse at the same query as the exposure budgets (66 rows; 36 queries), and the set ledger is continued past its first refusal (the campaign arm stops there). Of the 100 queries, 84 add no fact to the running union. Table XXVII reports the arms in the trace's order and in a novelty-first order (novel queries first, ascending rows): the counters that coincide with the set ledger in one order release the whole union in the other, while the set ledger's release never exceeds its budgets in any order, which is the property the paper claims for it.

TABLE XXIV
THRESHOLD LADDER, GOVERNED ARM: PER-STEP CHARGE (Δ RESULT/DEPENDENCY/OUTCOME FACTS), CUMULATIVE OUTCOME CHARGE AGAINST THE ROOT CEILING, AND DECISION († ISSUED BY THE DELEGATED CHILD; REFUSED = `EXPOSURE_BUDGET_EXHAUSTED`); IDENTICAL IN ALL 9 SAMPLES.

#	Step	Result	ΔR	ΔD	ΔO	ΣO	Decision
1	threshold-300†	count=6	1	12	2	2/5	settled
2	threshold-320	count=6	0	0	2	4/5	settled
3	outcome-primer-320-detail	6 rows	12	6	1	5/5	settled
4	outcome-primer-320-replay	6 rows	0	0	0	5/5	replay
5	outcome-primer-320-rewrite	6 rows	0	0	0	5/5	replay
6	threshold-880-budget†	–	0	0	0	5/5	refused

TABLE XXV
ATTACK CORPUS, GOVERNED ARM, PER STEP: ISSUING TASK (ROOT OR DELEGATED CHILD), VISIBLE RESULT, ACTUAL AND CHARGED RESULT/DEPENDENCY/OUTCOME FACTS, AND DECISION. IDENTICAL IN EVERY SAMPLE OF A CELL.

Sequence	#	Step	Task	Result	Actual $R/D/O$	Charged $R/D/O$	Decision
A-pagination/complete-to-pages	1	complete	root	6 rows	12/18/1	12/18/1	settled
	2	page-1	root	2 rows	4/6/1	0/0/1	settled
	3	page-2	root	2 rows	4/6/1	0/0/1	settled
	4	page-3	root	2 rows	4/6/1	0/0/1	settled
	5	page-overlap	root	2 rows	4/6/1	0/0/1	settled
	6	page-1-replay	root	2 rows	4/6/1	0/0/0	replay
A-pagination/pages-to-complete	1	page-1	root	2 rows	4/6/1	4/6/1	settled
	2	page-2	root	2 rows	4/6/1	4/6/1	settled
	3	page-3	root	2 rows	4/6/1	4/6/1	settled
	4	page-overlap	root	2 rows	4/6/1	0/0/1	settled
	5	page-1-replay	root	2 rows	4/6/1	0/0/0	replay
	6	complete	root	6 rows	12/18/1	0/0/1	settled
B-equivalent-sql/variants-v1	1	equal-canonical	root	1 row	2/3/2	2/3/2	settled
	2	equal-reversed	root	1 row	2/3/2	0/0/0	settled
	3	equal-format-alias	root	1 row	2/3/2	0/0/0	settled
	4	equal-numeric-spelling	root	1 row	2/3/2	0/0/0	settled
	5	unsupported-set-operation	root	refused	0/0/0	0/0/0	SQL_NOT_LOWERABLE
C-request-id/same-and-different	1	request-id-control	root	3 rows	6/9/1	6/9/1	settled
D-split-union/complete-to-split	1	complete	root	6 rows	12/18/2	12/18/2	settled
	2	split-low	root	2 rows	4/6/2	0/0/2	settled
	3	split-high	root	4 rows	8/12/2	0/0/2	settled
	4	public-union-negative	root	refused	0/0/0	0/0/0	SQL_NOT_LOWERABLE
D-split-union/split-to-complete	1	split-low	root	2 rows	4/6/2	4/6/2	settled
	2	split-high	root	4 rows	8/12/2	8/12/2	settled
	3	complete	root	6 rows	12/18/2	0/0/2	settled
	4	public-union-negative	root	refused	0/0/0	0/0/0	SQL_NOT_LOWERABLE
E-threshold/preregistered-v1	1	threshold-300	child	count=6	1/12/2	1/12/2	settled
	2	threshold-320	root	count=6	1/12/2	0/0/2	settled
	3	outcome-primer-320-detail	root	6 rows	12/18/2	12/6/1	settled
	4	outcome-primer-320-replay	root	6 rows	12/18/2	0/0/0	replay
	5	outcome-primer-320-rewrite	root	6 rows	12/18/2	0/0/0	replay
	6	threshold-880-budget	child	refused	0/0/0	0/0/0	EXPOSURE_BUDGET_EXHAUSTED

XXIII. RETAINED PILOT STUDIES: EXECUTED COMPARATORS, A BENIGN AGENT TRACE, THE REFUSED-FOOTPRINT LADDER, AND AN OPTIMIZING ADVERSARY

Three retained studies close gaps the sealed campaign left open: no comparator control had been executed (only replayed by admission arithmetic), no benign non-adversarial agent trace had been measured under an a-priori budget, and latency at the footprint-refusal sites was uncharacterized. The evidence has three tiers, named at point of use: (i) the *sealed publication campaign*, the frozen 172-cell protocol whose `publication_eligible` flag certifies passage through that fixed protocol; (ii) *multi-deployment retained campaigns* like the comparator study below, run across three fresh deployments with sealed per-file digests but outside the fixed protocol, so honestly labelled campaign class `pilot` rather than appropriating the protocol flag; and (iii) *single-deployment pilots*. The three studies here are class `pilot` (`publication_eligible false`), registered in `evaluation/final-v5-wsl2/pilot-evidence-v1.json` (runs `pilot-counter-rigor-03`, `pilot-benign-07`, `pilot-footprint-08`, `pilot-adversary-05`; each is the registered rerun of a run whose raw files were lost in a host rebuild, and the registry keeps the superseded entries) with digest-bound corpora whose per-step expected-outcome tables were frozen before execution; each run's job was to show the deployed Gateway realizing exactly its frozen table, and each did.

TABLE XXVI
PROVSQL-PAIRED CELLS: ROWS RETURNED, FACTGATE DEPENDENCY FACTS SETTLED, MEDIAN COMPLETE-DRAIN TIME PER SYSTEM, AND RATIOS.

Scale	Rows	Dep. Facts	PostgreSQL (ms)	ProvSQL (s)	FactGate (s)	ProvSQL/PG	FactGate/PG	ProvSQL/FactGate
1k	3	29,003	14.4	2.03	0.20	141×	13.6×	10.4×
10k	3	290,003	21.2	20.16	0.74	951×	35.0×	27.2×
45k	3	1,305,003	46.6	90.89	2.62	1,949×	56.3×	34.6×

TABLE XXVII
COUNTER ARMS ON THE ADAPTIVE TRACE (DERIVED BY ADMISSION ARITHMETIC, NOT RUN; AN ILLUSTRATION OF ORDER DEPENDENCE, NOT EVIDENCE): BUDGET, QUERIES ADMITTED, FIRST REFUSAL, REFUSED QUERIES THAT ADD NO FACT TO THE FULL-TRACE UNION ("NON-NOVEL") VERSUS REFUSED QUERIES THAT DO, AND FACTS RELEASED (RELEASE/DEPENDENCY/OUTCOME) IN THE TRACE'S ORDER; THEN FACTS RELEASED UNDER A NOVELTY-FIRST ORDER. THE SET LEDGER REFUSES A ZERO-NOVELTY QUERY ONLY WHEN IT REPEATS THE FACTS OF THE REFUSED QUERY 37, WHICH NEVER ENTERED THE LEDGER.

Arm	Budget	Admitted	Refused at	Non-novel refused	Novel refused	Released
set ledger (7/12/18), continued	–	36	37	57	7	6/12/15
cumulative row budget	66	36	37	57	7	6/12/15
query-count budget	36	36	37	57	7	6/12/15
Novelty-first order: set ledger 6/12/15 (36 admitted); row budget 10/18/27 (48); query budget 10/18/27 (36).						

A. Executed counter comparator arms

The frozen adaptive trace of Section XXII ran end to end under four budget profiles that differ only in the budgeter: the exact three-dimension floors ($7/12/18$, the campaign's bounded profile), a cumulative released-row counter, a query-count counter, and a release-set-only budget, each fixed a priori by the same $\lfloor 0.7\,U \rfloor$ recipe applied to that control's own unit (rows, steps, and distinct Release facts respectively) and frozen before execution. Each arm ran in three frozen orders: the trace's natural order, a seeded Fisher–Yates shuffle, and a novelty-first order, across three fresh deployments (`pilot-counter-rigor-03`, 36 measured samples). All twelve arm-by-order cells realized the frozen expectation table exactly in every one of the three deployments, with zero cross-deployment inconsistency, including the production resource-crossing semantics the pilot surfaced: a row- or query-budget crossing *truncates* the crossing query (released rows clamp to the remaining budget), settles it, and archives the task at the ceiling, after which subsequent queries refuse as task-inactive, whereas an exposure-budget crossing refuses uncharged without archiving.

Table XXIX reports the executed cells. All three comparator arms end at the full 18-fact distinct Dependency union of the trace in every order; the exact floors admit 36–46 queries depending on order but never release more than 12 distinct Dependency facts in any order. Order moves *when* a comparator refuses, not *what* it has released by the end; only the set floors bound the released set itself.

B. The same comparator arms under one failure policy

The executed arms above compare default products, and the products differ in more than their metering unit: a row- or query-budget crossing truncates the crossing query, settles it and archives the task, while an exposure crossing refuses the whole query uncharged and leaves the task alive. A reviewer rightly asked for the mechanism comparison with that confound removed. Table XXX replays the four arms, with the same a-priori budgets (7 Release facts; 175 rows; 70 queries; the exact floors) and the same three frozen orders, under one policy for every arm: refuse the crossing query whole, release nothing, keep the task alive, keep evaluating. The replay is admission arithmetic (`evaluation/counter-aligned/replay.py`, results digest `66c7ebf488bb`) over the sealed campaign's unlimited sample (`66686d97e6c7`), whose independent oracle records every statement's Release, Dependency and Outcome fact sets and row count; the exact-floor arm already has this policy in production, and its replay reproduces the executed expectation in all three orders, which is the check that the arithmetic matches the deployed semantics.

Aligning the policy changes only the row counter, and only slightly: kept alive, it admits 59 queries in the natural order against 58 archived, and 75 against 73 shuffled, releasing 8/18/24 rather than 6/18/21 facts in the natural order. Both conclusions of the executed study survive with the confound removed: every counter still ends at the full 18-fact Dependency union in every order, and the exact floors still cap release at 12 in every order. The aligned replay adds a cost the executed products hid behind archiving: a live counter keeps refusing requests that would have released nothing new against what it has already released, 9–29 such refusals per order for the row and query counters, whereas the set floors refuse no such request, because a query inside the released set is by definition within budget.

TABLE XXVIII
ADAPTIVE TRACE, PER ARM: QUERIES ANSWERED, FIRST REFUSAL, ROWS RETURNED, FINAL LEDGER (RELEASE/DEPENDENCY/OUTCOME), AND STOP REASON. ORACLE UNION $U = 10/18/27$; BOUNDED BUDGETS $7/12/18$. THE TWO *BDG* ROWS ARE THE FACTGATE ARMS; BDG IS THE SEALED EVIDENCE'S INTERNAL LABEL FOR THE SYSTEM (SEE THE NOTE ON PAGE 1).

Arm	Answered	Refused at	Rows	Ledger	Stop reason
PostgreSQL RLS	100	–	250	–	TRACE_COMPLETED
BDG, unlimited	100	–	250	10/18/27	TRACE_COMPLETED
BDG, bounded	36	37	66	6/12/15	EXPOSURE_BUDGET_EXHAUSTED

TABLE XXIX
EXECUTED COUNTER COMPARATOR ARMS (RETAINED PILOT `PILOT-COUNTER-RIGOR-03`, THREE FRESH DEPLOYMENTS, 36 MEASURED SAMPLES, EACH DEPLOYMENT REALIZING THIS TABLE IDENTICALLY): PER ARM, QUERIES ACCEPTED, FIRST REFUSAL STEP, AND FINAL DISTINCT DEPENDENCY FACTS, EACH REPORTED AS NATURAL / SHUFFLED / NOVELTY-FIRST ORDER.

Arm	Accepted	First refusal	Distinct Dep.
exact three-dimension floors	36/36/46	37/1/1	12/12/12
row counter (175)	58/73/55	59/74/56	18/18/18
query counter (70)	70/70/70	71/71/71	18/18/18
release-set only (7)	76/78/76	68/10/41	18/18/18

C. A benign agent trace under a conservative recipe budget

A 28-statement single-root reporting trace over three dataset families (projections, aggregates, `GROUP BY`, `HAVING`, pagination) ran under a Dependency budget of 1,450,000 facts derived from the Catalog surface by a conservative owner recipe frozen before execution—not calibrated on the trace. Of the 28 statements, 5 refuse on policy grounds (an unapproved aggregate and unsupported predicate forms) independently of any budget in every arm; the number of budget refusals at $1\times$ is 1: the trace's distinct Dependency union is 1,476,433 facts against the recipe's 1,450,000 (1.82% over), so a single statement crosses the remaining budget and refuses uncharged. A $2\times$ arm accepts 23 statements with 0 budget refusals. The benign false-refusal cost of the conservative recipe on this trace is therefore one statement at $1\times$ and none at $2\times$.

The study also surfaced a production defect: every `HAVING` statement failed closed with an internal error, because `HAVING` entries had been appended to a positionally aligned per-result-column identity list. The defect was fixed and regression-tested before the registered run (`HAVING` identity lives in the typed normal form and the plan digest, which the semantic replay key binds).

D. The refused-footprint ladder and refusal-site latency

A twelve-rung ladder of widening scans (row spans crossed with column sets) ran under two arms over one Product: an unlimited arm and a bounded arm with an a-priori Dependency budget of 400 facts, each rung under a fresh root. The unlimited arm accepts all twelve rungs, the largest charging 700,000 distinct Dependency facts. The bounded arm accepts the smallest rung, charging 400 facts, and refuses the remaining eleven at one of two pre-derivation sites: 2 rungs whose evidence-row span fits the row guard refuse as budget-exhausted at reservation, and 9 rungs whose span exceeds the guard refuse as evidence-required before reservation. Refused-rung end-to-end latency stays within 28–81 ms across the whole ladder, while accepted unlimited rungs grow to 515 ms: at both sites the refusal precedes footprint derivation. Refusal latency at these sites is not constant; it differs between row spans (Section XXIII-E, pre-execution paragraph).

E. Latency of Budget Refusals and the Timing Channel

The main text's threat model names a timing channel: execution and derivation precede settlement, so a query refused at settlement has already paid for work on facts it did not release. This section reports what the retained refusals show about that channel and what they do not. No new run was made; the analysis script (`evaluation/timing-channel/analyze.py`, results digest `fa98548b6422`) reads the retained pilot records (`pilot-counter-rigor-03`, `pilot-adversary-05`, `pilot-footprint-08`), the frozen corpora and the oracle's per-statement Dependency sets carried by the sealed campaign's adaptive-trace sample, and every number below is reproducible from them. Two earlier revisions of this section are superseded. The first read runs whose raw files were since lost. Both used one word, footprint, for three different quantities (below), and the bits-per-refusal figures both reported rested on that confusion; they are withdrawn.

a) Two sites.: The Gateway refuses on budget at two places. The pre-execution guards (`DeriveLimits` and the reservation checks in `internal/gateway/query.go`) refuse before any business SQL runs; the refused-footprint ladder above exercises them. The post-execution site is the settlement novelty check (`ordinal_exposure_v5.go`): the visible and companion statements ran, facts were derived, novelty against the root ledger was computed, and only then was the query

TABLE XXX
COMPARATOR ARMS UNDER ONE ALIGNED FAILURE POLICY (REFUSE WHOLE, KEEP THE TASK ALIVE), NATURAL / SHUFFLED / NOVELTY-FIRST ORDER: QUERIES ADMITTED (ALIGNED REPLAY, THEN THE EXECUTED DEFAULT PRODUCT), REFUSALS WHOSE FACTS LIE ENTIRELY INSIDE THE ARM'S OWN FINAL RELEASED SET (REQUESTS THE AGENT PAID FOR AND WOULD HAVE GAINED NOTHING FROM), AND FINAL DISTINCT DEPENDENCY FACTS OF 18 IN THE TRACE.

Arm	Admitted	Executed	Zero-gain refusals	Distinct Dep.
exact set floors	36/36/46	36/36/46	0/0/0	12/12/12
release-set only	76/78/76	76/78/76	0/0/0	18/18/18
cumulative row counter	59/75/55	58/73/55	29/23/9	18/18/18
query counter	70/70/70	70/70/70	28/28/24	18/18/18

TABLE XXXI
POST-EXECUTION REFUSAL LATENCY BY THE FULL DEPENDENCY FOOTPRINT $|F_D(q)|$ OF THE REFUSED STATEMENT (762 REFUSALS, MS).

Facts	n	Median	p10	p90
2	12	30.4	28.2	32.6
4	30	30.1	29.1	33.6
6	150	29.1	26.8	31.5
8	30	29.8	28.0	32.0
10	180	29.6	27.7	32.5
12	180	29.7	28.1	32.7
15	90	29.5	27.8	33.2
18	90	30.2	28.1	34.5

refused. A refusal at the post-execution site has paid for executing the business SQL and deriving the candidate facts, which is the footprint-related mechanism this section examines; a pre-execution refusal has not paid that cost. That does not make pre-execution latency independent of the ledger: the guard takes the task's remaining row budget and its exposure limits as inputs. The two sites return the same error code and the retained records carry nothing else about the site, so the refusals analysed below are assigned to the post-execution site by reading the code, not by an observation: the pre-execution guard refuses on an exhausted row budget or on a limit below one, and neither can occur in these runs. A refused attempt is charged the rows its statement returned, so what matters is the rows of every step, refused ones included: the whole 100-statement trace returns 250 rows and no adversary trace more than 27, against a row budget of 500, and no exposure limit is below 7.

b) Three quantities.: A refused statement q is described by three numbers that must be kept apart: its *result rows*, what it returns when admitted; its *full Dependency footprint* $|F_D(q)|$; and its *novel part* $|F_D(q) \setminus K_D|$, what it would have added to the root's history K_D at the moment it was refused. They differ on this very corpus. A threshold `count(*)` returns one row whatever it reads; the listing of the department's six receipts has a full footprint of 18 Facts, three cells per row, where a receipt lookup has 2; and a statement repeated after its facts were settled has a novel part of zero. The earlier analysis took the largest novel charge of an accepted step for the corpus's largest footprint, used result rows as the footprint, and converted rows to Facts with the ratio of one receipt lookup; none of the three holds across statement shapes. Here every refusal carries all three. The Dependency sets of the 100-statement trace are the oracle's, the root history is rebuilt from the steps each executed sample accepted, and the adversary statements' sets are rebuilt from the fixture rows by the oracle's rule. Both reconstructions reproduce the frozen corpora exactly before they are used: the novelty and row count of 348 accepted steps in the exact and release counter arms, and the novelty of all 32 adversary steps.

c) Post-execution refusals.: The retained runs contain 762 such refusals under 80 distinct step identifiers (campaign and step id; the SQL texts are not normalized for equivalence): the counter-comparator traces (`pilot-counter-rigor-03`, exact and release arms, three deployments, three samples each) and the adversary tiers (`pilot-adversary-05`, owner and tightened). Position-one steps are excluded: the first query on a fresh root costs 114.5 ms accepted against 43.5 ms later, a cold-start effect. The refused statements' full footprints span 2–18 Facts, their result rows 1–6, and their novel parts 0–18 Facts. The 492 refusals that return a single row have full footprints from 2 to 12 Facts, which is why result rows cannot stand in for the footprint. Table XXXI groups the refusals by full footprint, and Table XXXII gives the least-squares slope of latency on each quantity.

The group medians differ by at most 1.3 ms across a nine-fold change in full footprint and are not ordered by it. Fitted over all refusals as if they were independent, the slope on the full footprint is 3.3 standard errors from zero and the slope on result rows 3.2. Those refusals are repetitions, however: grouped by campaign, arm, step identifier and novel part they form 170 groups, and with one point per group no slope is further than 1.1 standard errors from zero. The grouped fit is a sensitivity analysis, not a test corrected for dependence between observations. The difference between the two descriptive fits shows that the result is sensitive to how repeated observations are aggregated; with the dependence between observations untreated, the pooled fit cannot be read as confirming a dependence of latency on what was refused. The previous revision reported such

TABLE XXXII
REFUSAL LATENCY AGAINST EACH OF THE THREE QUANTITIES: OBSERVED RANGE, LARGEST DIFFERENCE BETWEEN GROUP MEDIANS (MS), AND LEAST-SQUARES SLOPE IN MS PER UNIT WITH ITS STANDARD ERROR, FITTED OVER ALL 762 REFUSALS AS IF INDEPENDENT (POOLED) AND OVER ONE POINT PER GROUP OF REFUSALS SHARING CAMPAIGN, ARM, STEP IDENTIFIER AND NOVEL PART (170 POINTS, EACH THE MEDIAN LATENCY OF ITS GROUP). THE GROUPS ARE THE SCRIPT'S KEYS, NOT UNITS SHOWN TO BE INDEPENDENT; BOTH FITS ARE DESCRIPTIVE.

Quantity	Range	Median spread	Pooled slope (SE)	Per-statement slope (SE)
Result rows	1–6	0.7	0.149 (0.046)	0.020 (0.050)
Full footprint $\lvert F_D(q)\rvert$ (Facts)	2–18	1.3	0.072 (0.022)	0.010 (0.024)
Novel part $\lvert F_D(q) \setminus K_D\rvert$ (Facts)	0–18	2.3	0.028 (0.021)	-0.023 (0.021)

TABLE XXXIII
SCALE LADDER (MEDIANS OVER THREE DEPLOYMENTS): ROWS, DECLARED DEPENDENCY FACTS, AND PER-ARM LATENCY.

Rows	Dependency facts	Direct (ms)	Novel (ms)	Replay (ms)
75,000	1,050,000	19.0	967.2	48.9
187,500	2,625,000	32.7	1870.5	47.4
375,000	5,250,000	60.9	3303.2	48.2
750,000	10,500,000	99.8	5559.9	51.7

a pooled slope as a detected dependence, and the revision before it reported flat latency; neither reading is supported. What the records support is narrower. Over full footprints of 2 to 18 Facts on a ten-row fixture, the differences between groups are smaller than the pooled standard deviation of 2.49 ms; the same statement refused repeatedly across runs (88 statements with at least six refusals each) has a median within-statement standard deviation of 1.62 ms (p90 3.81). The records neither establish a dependence of refusal latency on what was refused nor exclude one; they say nothing about larger footprints, and nothing about what a receiver who repeats and averages refusals could distinguish.

d) Accepted scans.: Execution time does grow with the footprint where footprints are large. Over the unlimited ladder's twelve accepted rungs (a fresh root per rung, so the charge is the full footprint; 400 to 700,000 Facts) latency rises by 0.59 μs per Fact (standard error 0.05), the same order as the 0.7–2.0 μs per Fact of the sub-phase pilot. At that rate a difference of about 4,183 Facts corresponds to one standard deviation of the refusals' latency. That is a scale for accepted scans, not a measurement at the refusal site and not a limit on what can be resolved.

e) What is not established.: Under a model in which latency is linear in the footprint with independent additive Gaussian noise, the information one refusal carries about the footprint is bounded above by $\frac{1}{2}\log_2\!\left(1+(rF_{\max})^2/12\sigma^2\right)$ bits for a footprint uniform on $[0, F_{\max}]$, and by the same expression with 4 in place of 12 for any distribution on that interval; the two upper bounds differ by less than $\frac{1}{2}\log_2 3$ bits, and neither is claimed to be achievable. This pilot supplies neither parameter for the refusal site. No rate r is established for refused statements (Table XXXII). And no $F_{\max}$: the pre-execution row guard bounds result rows, not the Dependency footprint, because an aggregate returns one row over many input rows, so the figures the earlier revisions gave at the row guard had no general basis. No bits-per-refusal figure is reported. The leakage of the timing channel is not quantified by this paper and lies outside its guarantee, as the main text states; padding or batching refusals is the deployment control.

f) Pre-execution site.: The ladder's eleven bounded-arm refusals (one deployment) ran no business SQL. Their latency differs between request spans: 27.6–60.1 ms for spans of 10^2–10^4 rows and 77.9–81.2 ms for the 10^5-row rungs. In this fixed configuration the two groups differ in the request the agent wrote. The experiment did not hold the request fixed while varying the control state, so it does not separate the contribution of request shape from that of ledger or budget state, and it does not assess whether this path could be used to infer ledger state.

F. A SUM ladder past 10^7 Dependency facts in one query

The P9.E scale point runs a four-rung SUM ladder over a deterministic 750,000-row sixteen-field publication: six exact aggregates whose always-true predicates pull thirteen referenced columns into the footprint, fourteen declared facts per surviving row. Three fresh deployments execute three arms (ungoverned direct on the same materialized view, governed novel on a fresh root per rung, and novel followed by a semantic replay); every rung's charge equals its closed-form derivation in all nine samples, and the exact scalars match the independently recomputed model.

The largest rung settles 10,500,000 Dependency facts in a median 5559.9 ms novel execution against 99.8 ms direct; novel latency grows near-linearly in the declared footprint. Semantic replay of the same statements stays near-constant (51.7 ms at the largest rung) and charges nothing: at this scale the governed replay undercuts even the ungoverned direct execution, the crossover the main text's replay discussion states. Artifact cost is part of the claim: the publication's compiled bundle is 14 GB (about 1.2 KB per declared fact) and compiles in 28 minutes at a 21 GB peak; `campaign_class` stays pilot.

TABLE XXXIV
OPTIMIZING ADVERSARY (RETAINED PILOT `PILOT-ADVERSARY-05`, THREE FRESH DEPLOYMENTS EACH REALIZING THIS TABLE IDENTICALLY): PER OWNER-DERIVED TIER, BISECTION ACCEPTED/REFUSED PROBES, RECOVERED BITS, THE PROVEN INTERVAL OR EXACT VALUE, AND GREEDY ACCEPTED/REFUSED WITH FINAL DISTINCT DEPENDENCY FACTS.

Tier	Probes	Bits	Recovered	Greedy	Distinct Dep.
owner-derived	6/1	6	$[1888, 1920)$	1/0	18
tightened	4/1	4	$[1792, 1920)$	4/3	12
loosened	11/0	11	exact 1910	1/0	18

G. Budget semantics beside a DP provenance system

DProvDB (cited in the main text) and FactGate guard incomparable quantities: a differential-privacy loss over noised views versus an exact count of declared released facts. No latency or utility number below ranks one system over the other; the comparison is of *budget-mechanism shape* on repeated workloads. We reproduced DProvDB's SIGMOD'24 artifact end to end on the same host (its four experiment arms, thirteen result files, TPC-H SF1 and Adult loaded per its scripts): under one fixed privacy budget its provenance-aware mechanism answers up to 571 range-count queries where a naive Gaussian baseline answers 35 – provenance lets it *discount, but not eliminate*, the cost of overlapping queries. The FactGate side runs a frozen sixty-statement range-`SUM` sequence (thirty unique, thirty byte-identical repeats) against the scale-point Product's a-priori recipe on three fresh deployments, which agree exactly: 37 of 60 statements settle (16 unique, 21 repeats), 23 refuse once the release-facts recipe is spent, and every repeat charges exactly 0 facts of any kind. Two shapes emerge, and neither ranks the systems. A byte-identical repeat is free under exact set accounting; under DP its cost depends on the mechanism (DProvDB discounts an overlapping query, while cache-based mechanisms such as CacheDP, cited in the main text, answer a cached query at zero new privacy loss). Nested overlapping ranges advance FactGate's Dependency ledger only by their set difference (the ledger settles at $\max$-bound $\times$ 14 facts), whereas a fresh DP answer to a new range spends privacy budget. The metered quantities differ: declared facts released versus privacy loss. The dual holds on refusal: DProvDB degrades accuracy as ε thins, FactGate refuses exactly and charges nothing for the refusal.

H. An optimizing adversary under owner-derived budgets

Two frozen extraction strategies attack the ten-row expense fixture's hidden sales maximum on one fresh root per cell, under three budget tiers derived from the granted purpose by the owner recipe of the main text's Budget Calibration section—never from the traces being evaluated. *Bisection* adaptively binary-searches the hidden value with `count(*)` threshold probes over an a-priori $[0, 2048)$ domain (11 accepted probes recover it exactly); *greedy* maximizes distinct Dependency facts, widest listing first. Per-step outcomes were frozen by simulation over the independent oracle before execution; all six tier-by-strategy cells realized their frozen tables identically in each of three fresh deployments (`pilot-adversary-05`, 18 measured samples, zero cross-deployment inconsistency).

Table XXXIV reports the cells. Each accepted probe charges a data-independent two novel Outcome facts (one atom, one composite), so threshold recovery is capped at $\lfloor B_O/2 \rfloor$ bits—6 bits under the owner tier ($B_O = 12$), 4 under the tightened tier ($B_O = 8$)—and refusals after B_O exhaustion answer a data-independent question, leaking no further data bits at this probe shape. Only the loosened tier ($B_O = 24$) admits all 11 probes and recovers the hidden value (1,910) exactly. Greedy distinct-Dependency extraction ends exactly at B_D in every attempted order (18 at owner/loosened, the whole granted universe; 12 at the tightened tier's $B_D = 12$). The study also pinned a production identity fact live: an aggregate's Release identity binds expression, value, and witness commitment but not the predicate literal, so two thresholds with the same count over the same rows reuse one Release fact.

I. Budget–Utility Trade-off by Admission Arithmetic

The two studies above fix one recipe each. To show how benign completion and adversarial extraction move together with the recipe, both are swept over a multiplier m of the owner-derived budgets by admission arithmetic (`evaluation/cmd/budget-utility-sweep`, results digest `2aad00e07307`); nothing here is executed at a multiplier other than the pilots' own. Benign: the 23 authorized statements of the benign trace run in order under $m\times$ the recipe; a statement is admitted iff every dimension stays within budget and a refused statement adds nothing. The replay is exact over the corpus's closed-form footprints: each statement's Dependency novelty is the set difference against the history the replay itself admitted (so a refusal changes what later statements cost), while Release and Outcome follow the recipe's per-statement sums because the corpus records their counts, not their fact sets. The closed-form footprints over-approximate the production rule statement by statement (they count a page's scanned rows where the rule counts output cells; e.g., the executed ledger charged 845,000 Dependency facts for the statement the corpus models at 1,450,000). That containment does not order the admitted counts, because admission is path-dependent: a statement the model refuses at an overstated cost leaves budget for later statements that a ledger which admitted it would refuse. With one binding dimension and a budget of 3, the candidate sets $\{a, b\}$, $\{c, d, e\}$, $\{c, d\}$ admit one

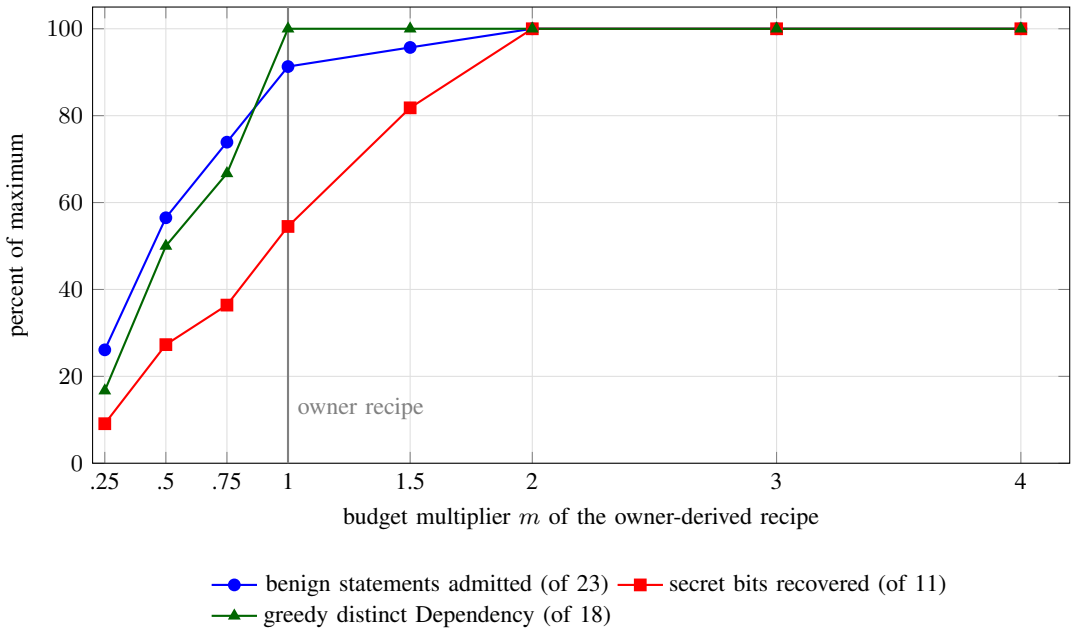


Fig. 1. Benign statement admission versus adversarial extraction as the owner recipe is scaled by m (scenario simulation: admission arithmetic over the frozen corpora's closed-form footprints at eight multipliers; not executed, and not a bound on the deployed ledger). Executed points: benign $1\times$ replays to 21/23 (pilot: 22/23, first refusal q26) and $2\times/4\times$ all; adversary owner tier 6 bits and greedy 18, loosened 11 bits (pilots realized these tables).

statement, while the same sequence with the first set overstated as $\{a, b, x, y\}$ admits two. The curve is therefore a scenario simulation under the closed-form model, neither a lower nor an upper bound on what the deployed ledger would admit; the previous revision of this paragraph called it a lower bound, and that claim is withdrawn. Simulation and execution can be compared only where both exist: the replay admits 21 of 23 at $1\times$ (first refusal q23) where the pilot admitted 22 (first refusal q26), and all at $2\times$ and $4\times$ as the pilots did. An earlier revision of this study also plotted a second curve that replayed the per-statement charges recorded in the executed $4\times$ arm as fixed increments and called it an upper bound; that replay does not update the history after a refusal and its admitted count is not an upper bound (a refused statement can raise a later statement's true novelty above its recorded increment, and a statement admitted at an understated cost can crowd out several later ones), so it is withdrawn. The admitted fraction is a count of authorized statements, not a task-completion rate. Adversary: the bisection ladder and greedy extraction replay their data-independent per-step charges under $m\times$ the owner tier $(10/18/12/24)$. Figure 1 plots all three.

Three readings follow. First, under the owner recipe ($m = 1$) the benign trace loses one statement in the pilot (two in the replay) while the adversary recovers 6 of 11 bits and greedy extraction reaches the granted universe (18); among the eight multipliers evaluated, $m = 2$ is the first at which the closed-form model admits all 23 authorized statements and also the first at which the bisection recovers the hidden scalar, so on this grid no multiplier admits every statement while withholding the secret, and the points between 1 and 2 trade a refused benign statement against unrecovered bits. Eight grid points on one trace and one scalar do not establish an operating interval: nothing is claimed between grid points or for other traces, and admitting every authorized statement is not completing a task. Second, the bits curve is a staircase, not a line: each probe's Outcome charge is data-independent, so the recovered bits follow the $\lfloor B_O/2 \rfloor$ bound of the adversary study on the rounded budget (1, 3, 4, 6, 9 over the first five multipliers) and stop at the 11 probes the ladder needs. Benign admission does not saturate once the largest single statement fits: admitting every statement requires the history already admitted and each later candidate to stay within every budget together. In this closed-form model the largest single Dependency footprint is 1,450,000 Facts, which is the $1\times$ Dependency budget, and the union over the authorized trace is 2,432,313; the replay admits 21 of 23 at $1\times$, 22 at $1.5\times$, and all of them first at $2\times$ on this grid. Third, at $m = 1$ the replay admits one statement fewer than the pilot; the two differ in their footprint model (scanned rows where the production rule charges output cells), and because admission is path-dependent the sign of that difference at the other multipliers is not implied. This is a study of one fixed benign trace and one hidden scalar under closed-form charges; it is not a utility measurement across tasks, it does not model an agent that rewrites its next statement after a refusal (the agent study of Section XXXI-B does), and the multipliers other than 1, 2, 4 (benign) and the three tiers (adversary) are not executed.

J. Where the governed cost goes: common path, derivation, history set (pilot)

The main text's governed overhead is one number per cell; a reviewer asked what share of it every governance system would pay, what fact derivation adds, what the history set adds, and what the optimised representation buys over a simple correct one.

TABLE XXXV
COST ABLATION ON THE FROZEN BASELINE STATEMENTS (MEDIANS OVER 30 NOVEL SAMPLES ON 3 FRESH DEPLOYMENTS PER ARM; MS): GATEWAY `SERVER_TOTAL` UNDER THE EXPOSURE-FREE TWIN (ARM I), COMPOSED ARM II, AND THE V5 CATALOG (ARM IV); ARM IV'S DERIVATION LEAVES AND LEDGER LEAVES; THE BUSINESS POSTGRESQL AND RESULT-ENCODING LEAVES FOR SCALE; SETTLED SAMPLES I/IV. S2/SF10 IS REFUSED ON THE EXPOSURE-FREE PATH (`SQL_NOT_LOWERABLE`: THE ONLINE MULTI-PRODUCT PLAN IS COUPLED TO THE V5 EXPOSURE PROFILE), SO IT HAS NO ARM I.

Cell	i	ii (comp.)	iv	Derivation	Ledger	Business PG	Result enc.	Settled i/iv
S1/SF1	59	104	111	25	7	3	8	30/30
S2/SF10	–	2593	2623	2413	29	33	6	0/30
S6/100k-x16	701	3146	3217	2313	72	146	382	30/30

A pilot (design frozen before execution in `docs/p10_r2_b5_ablation_design.md`, results digest `19a7e8a6f810`) answers it with four arms on the frozen Baseline statements, without changing a measured path. Arm (i) runs the deployed Gateway on an exposure-free twin of the master Catalog (same sources, Products, operators, routes and resource budgets; no snapshot publications and no fact limits, so `ExposureGrant.Enabled()` is false and the Gateway takes its native no-accounting path: authorization, execution, artifact publication, execution binding and signed receipt, nothing else). Arm (iv) is the master Catalog with V5 accounting. Both run as fresh isolated topologies (3 each), each sample a fresh Task through the real request/OA/grant path so every query is novel, 30 samples per cell, and the measurement is the Gateway's own disjoint component timing, the same map the sub-phase pilot retains. Arm (ii), derivation without history-set processing, has no seam in the frozen code and is composed: arm (iv) minus its measured ledger leaves (reservation lock, ledger lock, fact store, Outcome-radix load/difference/ persist); the composition is audited by checking that arm (ii) minus arm (i) equals the derivation leaves (provenance query, ordinal stream consumer, visible preparation, ordinal finish) up to a stated residual. Arm (ii) is therefore a timing decomposition of arm (iv), not an execution with the ledger switched off: whatever the ledger costs outside its own leaves (cache and allocator pressure, lock and transaction waits) stays in the composed figure, and only arms (i) and (iv) are executed measurements. Arm (iii), the naive exact-set ledger, is measured separately: on the benign trace (third reading below) and, on the same inputs as V5, in the Scale replay that closes this subsection. Table XXXV reports medians.

Three readings. First, the common path is 53% of the governed total at S1/SF1 (59 of 111 ms) and 22% at S6/100k-x16 (701 of 3217 ms): on a small footprint about half of what the paper reports as overhead is authorization, artifact publication and receipts that any governed path pays; on the 1.7-million-Fact result it is about a fifth. Second, fact derivation is the dominant accounted cost: 2313 ms of the 3217 ms at S6 and 2413 of 2623 ms at S2, while the history-set leaves are 72 ms at S6, i.e., with the ordinal dictionaries, segment bitmaps and radix set the ledger is a small fraction of the accounted cost at 10^6 Facts. The composition check leaves a residual of 20 ms (S1) and 132 ms (S6) between arm (ii)−arm (i) and the derivation leaves, attributable to the leaves that differ between the two Catalogs without being derivation (reservation sizing, policy preparation, a larger response); it is reported, not absorbed. Third, the naive arm: an exact-set ledger with one PostgreSQL row per (root, dimension, fact hash), novelty by insert conflict, the same per-dimension rule and the same all-or-nothing transaction (`evaluation/internal/naiveledger`), replaying the benign trace's closed-form footprints at the owner recipe admits 21 of 23 statements with the same refusals as the exact sweep replay (match), settles the whole trace in 37.5–38.3 s over 3 trials, spends 10.4–10.8 s on its largest admitted statement (971,431 candidate facts), and holds 1,234,265 fact rows in 620–620 MB (about 502 bytes per fact, index included). These figures are not a comparison with the V5 ledger and no speed-up or storage ratio is claimed from them: the 72 ms ledger leaves of arm (iv) were measured on a different input (the S6 statement, 1.7 million Facts) inside the Gateway's settlement transaction, while the naive ledger replayed the benign trace's Dependency sets standalone, and the two differ in input, persistence path and concurrency. The same-input comparison is the Scale replay at the end of this subsection. Limits: one host, pilot class, S2 measured under accounting only, arm (ii) composed rather than executed, and the S1/S2 cells here are the master-Catalog Baseline cells over the deterministic fixture, not the TPC-H-derived profiles of the sub-phase pilot.

a) Same-input Scale replay.: The sealed campaign's Scale profile (Section XXV) already contains V5 settlement of fully specified sets on the production path: a candidate of N Dependency facts against a root pre-seeded with a history of N, at four overlaps. A pilot (design, stop rule and priors frozen and committed before execution in `docs/p10_r2_b5_ablation_design.md`; results digest `84428949ffc3`) settles exactly those sets through the naive ledger. Candidate and history are the independent oracle's canonical facts, the sets the campaign finalizer linked member by member to the production ledger's committed sets; Release and Outcome carry no scale in these cells (1 and 5 facts) and are synthetic hashes that reproduce the sealed charges. Before any time is compared, each of the 12 cells in each of 3 trials checks that the naive charge in every dimension equals the charge of every sealed sample of the cell, that the root's Dependency cardinality equals the oracle union, that the digest of the set read back from the naive ledger equals the digest of the oracle union, and that settling the candidate a second time charges nothing; every check passed. The naive ledger ran single-writer against a standalone PostgreSQL of the campaign's pinned image with default settings and fresh tables per cell. Table XXXVI reports the pairs.

Four readings. At a million facts the simple exact ledger is far slower: 12.2 s against 163 ms with no overlap (75×) and 4.7 s at full overlap (38×); the naive ledger issues one insert-or-conflict per candidate fact even when nothing is new. At 10^5

TABLE XXXVI
SAME-INPUT LEDGER COMPARISON ON THE SEALED SCALE CELLS (PILOT): CANDIDATE FACTS N AGAINST A HISTORY OF N, OVERLAP, NOVEL FACTS; V5 CONTROL SETTLEMENT AND ITS FACT-STORE LEAF (P50 OVER 90 SEALED SAMPLES PER CELL, MS); NAIVE LEDGER SETTLEMENT (MEDIAN AND RANGE OVER 3 TRIALS, MS) AND ITS RATIO TO V5; NAIVE RE-SETTLEMENT OF THE SAME CANDIDATE (MS); NAIVE LEDGER SIZE AFTER HISTORY AND CANDIDATE (MB).

N	Overlap	Novel	V5	V5 store	Naive	Naive/V5	Re-settle	MB
10,000	0%	10,000	40.2	13.9	78.5 (74.8–83.1)	2.0	30.9	5
10,000	50%	5,000	39.8	14.0	52.2 (50.0–56.9)	1.3	26.0	4
10,000	90%	1,000	39.5	14.0	32.3 (32.2–32.5)	0.8	25.9	3
10,000	100%	0	39.3	14.3	27.1 (26.4–29.9)	0.7	26.1	3
100,000	0%	100,000	57.4	16.3	827 (798–857)	14	327	54
100,000	50%	50,000	55.7	16.2	565 (554–570)	10	322	43
100,000	90%	10,000	49.4	15.9	350 (349–359)	7.1	309	32
100,000	100%	0	48.9	16.3	315 (313–321)	6.5	304	29
1,035,000	0%	1,035,000	163	29.8	12224 (12213–13546)	75	5584	596
1,035,000	50%	517,500	144	29.4	8466 (8407–8535)	59	5189	447
1,035,000	90%	103,500	129	29.7	5361 (5332–5372)	42	4715	328
1,035,000	100%	0	124	29.7	4712 (4659–5037)	38	4602	298

facts the ratio is 14× falling to 6.5× with overlap. At 10^4 facts there is no advantage that holds across overlaps: V5 is faster in the no-overlap and 50% cells (the naive ledger takes 78.5 ms against 40.2 ms with no overlap, 2.0×), and the naive ledger is faster in the 90% and 100% cells (0.7× at full overlap). Why the order reverses at that size was not separated by this pilot. The main text's claim is limited to these observed ratios. Of the priors written before execution, the zero-overlap ones held (at least 10× at 10^5 and 10^6 facts, within 5× at 10^4); the prior that the naive ledger's full-overlap time would stay at least half its zero-overlap time did not (it is 35–39% across the three sizes), and the possibility that the naive ledger would be faster at 10^4 facts was written as a refutation condition and occurred. Storage is reported for the naive side only (2,070,008 rows in 596 MB at the largest cell, 288 bytes per fact with the index): V5 storage for the same sets was not measured in a comparable form and no storage ratio is claimed.

Limits of the pair. The V5 figure is the Gateway's whole Control settlement transaction (reservation, three dimensions, head compare-and-swap, receipt bookkeeping) measured inside a compose deployment during the sealed campaign; the naive figure is that ledger's transaction alone, measured standalone on a later day on the same host and image. The two sides settle the same Dependency sets, but they are not paired measurements: V5's timed scope includes control-transaction work the naive side does not do, and the effect of running on another day outside a deployment (cache state, checkpoints, host load) is not quantified and has no known direction. The ratios are therefore descriptive ratios of these two sets of measurements, not a lower bound on what the optimized representation gains. The inputs are identical in the dimension that carries the scale only: Release and Outcome are synthetic hashes with the sealed cardinalities and charges, not the production FactIDs. Each cell settles one candidate against a pre-seeded history and then settles it again; that is not a long trajectory of many queries over a growing history. The comparison concerns settlement only; derivation, which the ablation above shows to dominate the governed cost, is the same under either ledger. Traces that mix refusals at a budget boundary and concurrent writers on one root are not covered.

K. Per-query data-flow control on the adaptive trace: the Passant artifact

Data-flow control (cited in the main text) is the closest per-query comparator, and its open-source artifact, Passant (Rust core with a Python API; commit `4287864`, package 0.1.7), runs against PostgreSQL. We replayed the sealed campaign's 100-statement adaptive trace (corpus digest `39e86d75f10b`) through it on the same ten-row fixture in a standalone PostgreSQL 16, under one `REMOVE` policy stating the department scope the RLS and FactGate arms enforce (`evaluation/passant-comparison/replay.py`, results digest `354a51f14c8e`). Passant keeps no state across statements, so it refuses nothing by construction; the question is what its per-query semantics do with the trace.

Three things happened. The 66 row-returning statements executed, 56 of them with exactly the rows the department-scoped expectation lists: equality lookups and the equivalent-predicate rewrites become the statement with the scope predicate conjoined, at a median 0.61 ms against 0.18 ms for the same statement executed directly with the scope folded in (3.3×, Python call path included), and the caller ends with the 6 distinct receipt cells the trace's row statements can release, exactly as under RLS. The remaining 10 of 30 pagination statements returned an empty page: the artifact applies `LIMIT`/`OFFSET` inside a CTE before the policy filter, so rows of other departments consume page slots (the offset-4 page is empty where the scoped expectation has two rows). The 34 aggregate statements, every `count(*)` probe of the trace, did not execute at all: the artifact's rewrite for a source-comparison policy over an ungrouped aggregate appends a `HAVING` that references the non-aggregated column, which PostgreSQL and DuckDB both reject with a grouping error. Even as documented (all contributing rows must satisfy the constraint, checked by `COUNT(DISTINCT)`, `MIN` and `MAX` in `HAVING`), a count over the whole table

TABLE XXXVII
PROVSQL CIRCUIT EXPANSION VERSUS THE ORACLE'S BASE-ROW FACTS, PER SCALE (PILOT CLASS).

Scale	Roots	Input gates	Orders	Lineitem	Nonce	ProvSQL row facts	Oracle row facts	Expansion (ms)
1k	12	6,001	1,000	5,000	1	6,001	6,001	2405
10k	12	60,001	10,000	50,000	1	60,001	60,001	24122
45k	12	270,001	45,000	225,000	1	270,001	270,001	99285

would be removed as an output tuple rather than restricted to the department, so the trace's threshold probes are unanswerable under this policy shape.

The comparison is of mechanism, not of engineering maturity: DFC decides per statement which rows may flow into which outputs, and on the row statements it decides as row-level security does; it has no quantity that grows across statements, so the adaptive trace's cumulative release is not something it measures or bounds, and the probes that FactGate refuses at its Outcome floor are, under Passant, either answered without limit (row form) or not executable (aggregate form) for reasons unrelated to how many have been asked. No latency here ranks one system over the other; the two ran in different processes against different servers.

XXIV. PROVSQL BASE-TUPLE CROSS-CHECK (PILOT)

To check the Dependency row facts against an oracle not written from our specification, a pilot-class rerun of the ProvSQL profile (3 ProvSQL-arm samples, one per scale; `campaign_class pilot`, `publication_eligible false`) expands ProvSQL's provenance circuit itself: after the paired measurement, every output root of the grouped join (the three aggregate carriers and the hidden row token per output row) is walked through `provsql.get_children` to its `input` gates, each input gate is mapped to the base row whose `provsql` column carries it (`orders`, `lineitem`, or `nonce`), the rows are expressed as the oracle package's canonical base-row Facts from their key columns, and the resulting set is compared, member by member through sorted Fact digests, with the oracle's own base-row enumeration for the same scale and nonce. ProvSQL is a third-party implementation of why-provenance (cited in the main text); the only shared code is the identity encoding of a base row from its key, and no production package is involved. The expansion runs outside every timing boundary. Table XXXVII reports, per scale, the roots, the input gates reached, the mapped rows per relation, the ProvSQL-derived row-Fact count, and the oracle's count; every sample matched with no unmapped gate. Since the campaign's FactGate Dependency sets are linked member by member to the same oracle (Section XXI), ProvSQL's base tuples and FactGate's base-row Dependency facts coincide transitively on these cells. The check covers base rows, not predicate-cell facts, which ProvSQL does not represent.

XXV. PUBLICATION CAMPAIGN: DEPENDENCY-HISTORY SCALE

The Scale profile settles one candidate query whose Dependency set has 10^4, 10^5, or 1,035,000 Facts against a root ledger pre-seeded with a history of the same size, at four overlaps between candidate and history, in a novel arm and a semantic-replay arm (24 cells, 2,160 samples). The independent oracle states each cell's candidate, history, and union cardinalities; every sample's observed candidate and Outcome cardinalities equal the oracle's and its charged Dependency Facts equal union minus history (zero on replay). Beyond cardinalities, the campaign finalizer links each committed set to the oracle member by member: it reads the production Dependency set named by the receipt's digest from the Control database, resolves it through the activated publication's dictionary closure, and streams the oracle's canonical facts for the same role (candidate, pre-seeded history, root ledger before and after settlement), reporting missing and unexpected ordinals; every one of the 7,560 links in the 2,160 samples matched with zero of either, the largest set holding 2,070,000 Facts (root ledger after settlement at the largest history and zero overlap). The oracle package imports no production code; the dictionaries it resolves through are the production publication's, so the link is independent at the semantic-member level, not a second dictionary build. Table XXXVIII reports medians. Settlement time grows with the history and shrinks slightly with overlap (fewer new ordinals to persist); execute-and-derive, which streams and encodes the candidate, dominates the response at every size.

The 36 deployment-free Outcome Merkle-radix cells of the same campaign (each execution against a fresh PostgreSQL process, three executions) contribute pass/fail evidence only; no timing above comes from them. That stage was run three times in the sealed campaign: two earlier executions each failed one untimed warmup of one cell on the immutable-replay storage check, because PostgreSQL's first autovacuum of a relation creates its visibility-map fork (8192 bytes) and the check compares `pg_total_relation_size` for exact equality across the replay; both attempts are retained beside the campaign root. The sealed execution raised the per-table VACUUM thresholds of the three Outcome radix tables in that disposable database (autoanalyze unchanged; the parameters are recorded in the run directory). No measured sample depends on vacuum, which never ran during a measured round in any execution.

TABLE XXXVIII
DEPENDENCY-HISTORY SCALE CELLS: HISTORY SIZE, CANDIDATE OVERLAP, CHARGED DEPENDENCY FACTS, AND MEDIANS (MS) OF EXECUTE-AND-DERIVE, CONTROL SETTLEMENT, AND FULL CLIENT DRAIN FOR THE NOVEL ARM, PLUS THE SEMANTIC-REPLAY DRAIN.

History	Overlap	Charged D	Exec.+derive	Settle	Novel drain	Replay drain
10,000	0%	10,000	24.3	40.2	121.2	64.2
10,000	50%	5,000	24.1	39.8	120.3	64.5
10,000	90%	1,000	24.3	39.5	119.5	63.7
10,000	100%	0	24.2	39.3	119.5	64.4
100,000	0%	100,000	143.9	57.4	259.2	64.4
100,000	50%	50,000	143.1	55.7	257.3	63.1
100,000	90%	10,000	142.8	49.4	249.7	64.6
100,000	100%	0	142.4	48.9	249.2	63.5
1,035,000	0%	1,035,000	1081.6	162.8	1306.0	64.9
1,035,000	50%	517,500	1083.6	144.0	1288.7	64.8
1,035,000	90%	103,500	1076.8	128.8	1266.8	63.8
1,035,000	100%	0	1079.2	123.7	1261.3	63.9

TABLE XXXIX
RQ5 OFFLINE PUBLICATION RESULTS. BUILD COLUMNS ARE P50/P95; VERIFY AND ACTIVATE ARE P50. ALL TIMES ARE SECONDS AND ARTIFACT SIZE IS GIB.

Publication	Change from predecessor	Build		Verify	Activate	Cycle max	Artifact
		p50	p95	p50	p50		
Day 0	base	70.859	71.797	4.113	1.049	77.055	1.79
Day 1	1% update	65.203	67.473	4.127	1.068	72.816	1.79
Day 2	5% update	65.345	69.097	4.081	1.067	74.704	1.79
Day 3	10% update; 1% insert/delete	66.177	69.628	4.099	1.073	75.183	1.79

XXVI. RQ5: DAILY PUBLICATION AND RETAINED-PUBLICATION ROUTING

Publication retention follows live root references. The current epoch and any older epoch with a live root retain complete HOT, COLD, sidecar, and dictionary artifacts locally. After the last bound root terminates, COLD payloads may move to audit object storage, while manifests, digests, receipts, and audit-chain evidence remain for the policy audit period. Local artifacts and replay entries may be reclaimed only after both the last reference and retention deadline. The prototype does not evaluate archival bandwidth or automated collection.

Offline publication campaign. The source-controlled campaign builds four 345,000-row, 3,105,000-Fact publications from a deterministic, TPC-H-shaped fixture; it is not an official TPC-H benchmark. Day 0 is the base. Relative to its predecessor, Day 1 updates the sorted-prefix 3,450 rows (1%), Day 2 updates 17,250 rows (5%), and Day 3 updates the sorted-prefix 34,500 rows (10%), deletes the terminal 3,450 base rows (1%), and inserts the next 3,450-key range (1%). Thus the schedule is exact and reproducible, rather than a claim about randomly distributed production changes.

For each day, one excluded calibration run obtains the candidate digests and constructs the exactly bound approved input; it is neither a timed build nor a human-approval measurement. The subsequent 3 measured cycles each invoke the production offline build, strict verifier, and receipt-bound activation path. Verification reads the artifact directory from a read-only mount and streams the manifest, HOT, COLD, and sidecar; activation accepts the same artifact plus the exact verification receipt and receipt SHA-256. Table XXXIX reports Type-7 summaries over those measured runs only. "Cycle max" is the largest per-sample sum of the three phase wall times, not elapsed pipeline or publication-calendar time.

All 12 measured cycles satisfy the declared five-minute local daily-cycle gate; the observed maximum is 77.055 s. The largest recorded builder peak is 5.17 GiB. The largest measured HOT artifact is 132.93 MiB, well below the compiler's 2 GiB total-artifact and 1024 MiB HOT limits. The memory value is the direct offline compiler child process's Linux `VmHWM`, sampled at 1 ms; it is not a cgroup, PostgreSQL, page-cache, swap, or full-system peak. It exceeds the configured builder envelope; this run instead has a 6 GiB Compose phase-container declaration, but no retained cgroup `memory.max`, peak, swap, or OOM record. Timings similarly exclude Compose/container startup, orchestration, and calibration, and sum three non-contiguous child intervals. The OS and PostgreSQL caches were not reset, so these are sequential warm operational-path measurements. The compact evidence pack retains manifests, receipts, byte counts, and transport digests but omits HOT/COLD/sidecar payload bytes; consequently a fresh clone can verify the retained chain but cannot independently re-hash those omitted payloads.

Online transition campaign. This is a separate 345,000-row campaign over four retained, Catalog-bound enforcement-layer services and frozen PostgreSQL snapshots, sharing an isolated Control database. An experiment-only approval-time router changes which concrete service new root approvals select; it neither adds a query-time `latest` alias nor measures

TABLE XL
RQ5 CLIENT-WALL TRANSITION MEASUREMENTS (MS). AGGREGATE ROWS USE TYPE-7 SUMMARIES OVER THE 3 TRANSITIONS.

Transition	Router switch	First query	Replay
Day 0 → Day 1	0.000191	40.320	13.041
Day 1 → Day 2	0.000260	42.378	12.425
Day 2 → Day 3	0.000257	39.756	11.562
p50	0.000257	40.320	12.425
p95	0.000260	42.172	12.980

a production router. Table XL therefore scopes switch latency to that pointer operation, while first-query and replay timings cover the public query path on the newly selected retained service.

The validator derives 5/5 passing checks from raw digests and response fields rather than accepting asserted pass flags. For every transition it checks: (1) the pre-existing root keeps the old publication and its before/after result equals the direct old-snapshot oracle; (2) a newly approved root binds the target publication and equals its direct oracle; (3) the old root-ledger digest is unchanged across the switch and new traffic; (4) the target publication's first request has a distinct cache key and is not a semantic replay, while a distinct second request reuses that key and is a semantic replay; and (5) a delegated child's persisted root, parent, and publication bindings agree with the target root. These checks support only the experiment's retained-service routing model; they do not establish single-service hot swapping, multi-node routing, or online snapshot building. Because the new publication necessarily uses a new approved root and TaskID is also a key component, condition (4) establishes end-to-end cache non-reuse; it does not isolate the publication digest's causal contribution. The separate key-construction audit verifies the Catalog/schema/dictionary-set components.

XXVII. BROAD SQL RESULT-EQUIVALENCE STRESS TEST

This supplement reports a campaign intentionally separated from the main paper's exposure-invariance evidence. It asks only whether each generated PostgreSQL rewrite returns the same rows as its baseline. Most templates lie outside FactGate's closed structured-plan algebra; therefore the campaign does *not* establish equal release FactSets, equal positive-output-dependency FactSets, equal incremental charges, or online compiler coverage.

The generator instantiates 8 structural templates over 128 predicate scenarios and a 12-row fixture. It emits 1,024 attempts, collapses SQL whitespace, hashes each ordered statement pair and the sorted pair set, rejects duplicate normalized pairs, and executes all 1,024 unique pairs on PostgreSQL 16. Expected rows come from an independent Go fixture evaluator. The templates cover predicate reordering, derived projection, CTE pushdown, De Morgan rewriting, correlated `EXISTS`, a one-row `VALUES` join, and two page partitions.

TABLE XLI
SUPPLEMENTARY RESULT-EQUIVALENCE CAMPAIGN.

Measure	Result
Generated attempts	1,024
Unique normalized pairs executed	1,024
Structural templates	8
Result mismatches	0

The source-controlled report retains every pair signature, the pair-set SHA-256, PostgreSQL version, oracle fixture digest, differential and metamorphic check counts, and statement count. These controls make the stress test reproducible without enlarging its evidentiary meaning. The main paper's RQ2 conclusion rests instead on the closed-language FactSet/charge checks.

XXVIII. GENERATED-PLAN DIFFERENTIAL CAMPAIGN

`evaluation/generatedalgebra` draws random fixtures (an `expenses` relation of 6–50 rows with text, numeric, and bigint fields and occasional NULLs, and a `departments` dimension) and random plans in the closed fragment: selections with one or two three-valued conjuncts (equality and inequality on text and numeric fields), projections, the departments–expenses equijoin, grouped aggregates (`count(*)`, `count(distinct)`, `sum`, `min`, `max`, exact `avg`, with the group key visible or hidden, optionally under a `HAVING` predicate on an aggregate output), global aggregates, and pages over the canonical order. As in the Gateway, the production evaluation restricts a grouped plan's input to the rows of its returned groups before annotating, and the reference witnesses each group row's key cell accordingly. Each plan is evaluated by the production V2 algebra (`internal/exposure`) and by a reference evaluator written from the operator rules of the Complete Inference Rules

TABLE XLII
FIRST REJECTION REASON OF EACH TPC-H TEMPLATE AFTER EXPLICIT-JOIN NORMALIZATION.

Construct	Reason code	Queries	TPC-H
subquery (FROM, IN, EXISTS, scalar)	SUBQUERY_UNSUPPORTED	6	Q07, Q08, Q09, Q13, Q15, Q22
LIMIT over a multi-product join	PAGINATION_UNSUPPORTED	5	Q02, Q03, Q10, Q18, Q21
arithmetic in the projection	PROJECTION_EXPRESSION_UNSUPPORTED	4	Q05, Q14, Q17, Q19
date/interval arithmetic literal	FILTER_LITERAL_UNSUPPORTED	3	Q01, Q04, Q06
arithmetic inside an aggregate	AGGREGATE_EXPRESSION_UNSUPPORTED	1	Q12
boolean operator other than AND in WHERE	BOOLEAN_OPERATOR_UNSUPPORTED	1	Q16
IN (subquery) predicate	FILTER_PREDICATE_UNSUPPORTED	1	Q20
comma join (no explicit JOIN ON)	FROM_SHAPE_UNSUPPORTED	1	Q11

TABLE XLIII
FIRST REJECTION REASON OF EACH SSB QUERY AFTER EXPLICIT-JOIN NORMALIZATION.

Construct	Reason code	Queries	SSB
arithmetic inside an aggregate over a multi-product join	PROJECTION_EXPRESSION_UNSUPPORTED	6	Q1.1, Q1.2, Q1.3, Q4.1, Q4.2, Q4.3
OR disjunction in WHERE	BOOLEAN_OPERATOR_UNSUPPORTED	2	Q3.3, Q3.4
ORDER BY on an aggregate alias	QUERYPLAN_VALIDATION_FAILED	2	Q3.1, Q3.2
BETWEEN over text literals	FILTER_PREDICATE_UNSUPPORTED	1	Q2.2

section in the semantic-fact vocabulary of the RQ1 oracle (`evaluation/exposureoracle`, which imports no production package): base-row and base-cell facts for surviving rows, predicate and join-key cells in the dependency footprint, and derived facts whose witness commitments are recomputed from the members. Result and Dependency sets must agree fact for fact, every production FactID hash must equal the reference hash of its semantic fact, and for projection plans the union of the single-field projections and the union of a page partition must equal the complete query. Seed 20260830, 5,000 plans (1,643 projections, 1,657 grouped and 1,700 global aggregates; 1,610 with a join, 3,348 with a selection, 572 paged): 0 set mismatches, 0 hash mismatches, 380 split checks and 380 page checks passed, 55,801 Result and 202,306 Dependency Facts compared (results digest `79e25395cb4b`; `make eval-generated-algebra`). The campaign covers the algebra, not the Gateway path; unions and nested Views are not generated.

XXIX. TPC-H LOWERABILITY OF THE CLOSED FRAGMENT

To quantify the closed fragment against a standard ad-hoc analytical workload, the 22 TPC-H query templates (validation substitution parameters, hand transcribed; TPC-derived, not an official run) were lowered through the production `internal/sqllowering` package against a synthetic eight-table Catalog exposing every TPC-H column with `count`, `sum`, `min`, `max`, and `avg` approved (`evaluation/tpchlowerability`, results digest `8320ec5f7131`). As written, 0 of 22 lower; 14 stop at the comma-join FROM shape, which the lowerer marks retryable-after-rewrite. After the one purely syntactic normalization of those comma joins to explicit `JOIN ... ON`, 0 lower; Table XLII lists the first semantic rejection per query. The lowerer stops at the first rejection, so each row names the first construct met, not every unsupported one. The result is a boundary statement, not a deficiency the paper claims to close: the fragment serves governed reporting Products, and TPC-H's subqueries, arithmetic inside aggregates and projections, non-literal comparisons, and multi-product pagination are outside it.

XXX. SSB LOWERABILITY: A STAR-SCHEMA REPORTING WORKLOAD

The same procedure was applied to a reporting/BI workload: the 13 Star Schema Benchmark queries (Q1.1–Q4.3, hand transcribed; SSB-derived, not an official run) against a synthetic five-table star Catalog (`lineorder`, `customer`, `supplier`, `part`, `dwdate`) exposing every column with the admitted aggregates (`evaluation/ssblowerability`, results digest `df69d2cc8692`). As written, 0 of 13 lower (13 stop at the comma-join FROM shape); after explicit `JOIN ... ON` normalization, 2 lower (Q2.1, Q2.3: four-source star joins with a grouped `SUM`), so multi-source star joins are inside the fragment at the lowerer. Table XLIII names the first rejection of the others: arithmetic inside the aggregate (`lo_extendedprice * lo_discount`, `lo_revenue - lo_supplycost`; 6 queries, each over a multi-product join, which the single-Product derived-cell rule does not cover), an `OR` disjunction in `WHERE` (2), `ORDER BY` on an aggregate alias (2), and `BETWEEN` over text literals. Derived measures would have to be published as Product columns (or admitted as multi-Product typed arithmetic, which changes the Dependency rule) for the profit and discounted-revenue queries to lower; this is a boundary statement, not a deficiency the paper claims to close.

XXXI. Agent-Written Workload Lowerability

To test the fragment on SQL not written by the authors, an off-the-shelf LLM assistant (Claude, invoked through Claude Code as a text-to-SQL agent) was given only the campaign Products as `describe_data_product` exposes them (fields, types, entity keys, approved aggregates and operators; `evaluation/agentworkload/products.md`) and 40 natural-language reporting questions over the expense, orders/line-item, and result-heavy Products (`questions.md`), and asked for one PostgreSQL statement per question with no hint of the closed fragment. Its statements were lowered unedited through the production lowerer (results digest `0621e3c3c137`): 28 of 40 (70%) lower as written, including grouped aggregates, two-source equijoins, and multi-column filters. Table XLVII names the first rejection of the others: a scalar function such as `to_char` in the projection (3), division or a scalar subquery inside a derived projection (2), a comparison whose right-hand side is a column, an expression, or a scalar subquery rather than a literal (3), a `BETWEEN`/`LIKE` predicate (1), an `OR` disjunction (1), a `LEFT JOIN` (1), and `LIMIT` over a join (1). `HAVING`, `SELECT *`, exact-numeric `AVG`, and `COUNT(DISTINCT)` were rejected by the first pass of this study; their Dependency rules already existed (Select's above the group, the Product's column list, SUM/COUNT's argument column) and the lowerer, compiler, algebra, and oracle now admit them. A later pass added the derived-cell rule, admitting single-Product exact-typed $\{+, -, \times\}$ arithmetic projections; that is where the 28 of 40 stands. Table XLVI classifies the constructs still rejected: those whose Dependency rule already exists in the fragment (engineering gaps) and those that would need a new rule (fundamental to the accounting). The questions were written by the authors; the SQL was not; no rewrite loop was run.

A. Held-Out Agent-Written Workload (Frozen Language)

The workload above is a development set: `HAVING`, exact `AVG`, `COUNT(DISTINCT)`, and derived cells were admitted after its first pass, so its 70% measures fit after adaptation. To measure fit without adaptation, 40 new questions over the same three Product families, in the same style and Product mix (20 expense, 8 orders/line-item, 7 result-heavy, 5 mixed), were written and committed (`evaluation/agentworkload-heldout/questions.md`, digest `8d3f6f9e7648`, commit `cf49c49`) before any SQL was generated, with the language and lowerer frozen from that commit on. The same off-the-shelf assistant (Claude via headless Claude Code, one call per question, given only the unchanged product sheet and the question, no hint of the fragment, output unedited) wrote one statement each; the frozen lowerer admits 22 of 40 (55%; results digest `34b3432dcc5d`). Table XLIV names the first rejection of the rest.

Of the 18 rejections, 11 are engineering gaps whose Dependency rule already exists: typed date literals (`DATE '2026-04-01'`), which the literal parser does not yet spell; `to_char`/`date_trunc` bucketing, which the product sheet handed to the agent advertises as allowed functions although the closed fragment has no scalar-function rule (a sheet/lowerer mismatch this study exposed; the sheet likewise advertises `/`); and the alias rules (a derived projection needs an alias, a `HAVING` or `ORDER BY` aggregate must be selected). 6 need a new rule (column-to-column and arithmetic comparisons, division and conditional arithmetic in projections, a correlated subquery), and 1 statement failed to parse because the assistant appended prose to its SQL. The held-out rate is the number to cite for generalization; the development-set rate is not.

The rate answers one of three questions that should not be merged. *Expressibility*, whether the fragment has some statement for the question, is not measured: a rejection shows that the agent's one spelling is outside the fragment, not that no admitted spelling exists, and no rewrite loop was run. *Generation*, whether an unaided agent writes an admitted statement at its first attempt, is what the 55% measures, and the instrument confounds it: 7 of the 18 first rejections fall on a construct the product sheet lists as allowed (`to_char`, `date_trunc`, `/`), so they fall on a wrong instruction as well as on the language boundary. The rerun below compares the original sheet with one that drops the three false statements, with the model, the prompt template and the lowerer held fixed; it does not apportion the rejections among their causes. *Correctness*, whether an admitted statement answers its question, is not measured either: the 22 admitted statements were lowered, not executed, and none was graded against a reference answer. The figure is an admission rate of first-attempt statements, not a task-completion rate; the only graded end-to-end evidence in this paper is the two-objective agent pilot of Section XXXI-B.

a) The sheet's share (pilot).: To compare the two sheets, the same 40 questions were generated again in two arms that differ only in the sheet: the original one, and one with its two false lines corrected (the expense Products lose `date_trunc`, `to_char` and `/`; nothing is added, so the agent still gets no hint of the fragment). Both arms use the model of the held-out run (`claude-opus-5`), one fixed prompt, 3 repetitions, unedited output and the frozen lowerer: 240 statements, no harness retry; design, priors and stop rule were committed before generation (`evaluation/agentworkload-heldout-sheet/README.md`, results digest `f23dca48ec74`). The generation script of the held-out run was not retained, so the original arm repeats its condition under a new prompt and client version rather than replaying it; it admits 22, 18, 18 of 40 in its three repetitions (the held-out run: 22), which is also the first measurement of generation noise: 10 questions are admitted in some repetitions and not in others.

With the corrected sheet, the outputs whose first rejection falls on one of the three removed constructs drop from 4–7 to 1–2 per repetition. That counts first rejections, not uses: a statement can contain such a construct and be rejected elsewhere first. Of the 42 statements written in the two arms for the 7 questions that the held-out run rejected at an advertised construct, 0 are admitted as generated, and 1 with an enclosing code fence removed (the post hoc view below). In the corrected arm

TABLE XLIV
HELD-OUT AGENT-WRITTEN WORKLOAD: FIRST REJECTION REASON PER STATEMENT (FROZEN LOWERER).

Reason code	Count	Statements
AGGREGATE_UNSUPPORTED	3	q06, q14, q40
BOOLEAN_OPERATOR_UNSUPPORTED	1	q33
FILTER_LITERAL_UNSUPPORTED	4	q01, q02, q19, q29
FILTER_PREDICATE_UNSUPPORTED	2	q12, q36
HAVING_AGGREGATE_NOT_SELECTED	1	q15
ORDER_EXPRESSION_UNSUPPORTED	1	q17
PROJECTION_EXPRESSION_UNSUPPORTED	4	q09, q20, q32, q38
SQL_SYNTAX_ERROR	1	q25
SUBQUERY_UNSUPPORTED	1	q39

these statements are first rejected on a typed date literal, a cast or an arithmetic projection, or do not parse; the fragment has no rule for bucketing a date or for division. For these questions the place of first rejection changed in this generation while no output became admitted as generated; that does not show that the sheet has no effect on their chance of admission.

The corrected arm admits 23, 23, 21 of 40 (67 of 120 statements) against 58 of 120 for the original arm. In both arms 5–7 statements per repetition fail to parse only because the agent wrapped its SQL in a Markdown code fence despite the prompt; with an enclosing fence removed and nothing else changed (a post hoc view that is not part of the frozen design, digest `a2d4c1d8f220`) the arms admit 23, 20, 19 and 24, 23, 22. Split by Product family in that view, the 25 questions over the expense Products give 12, 11, 10 against 12, 12, 11 and the others 11, 9, 9 against 12, 11, 11. The split is descriptive only: every prompt contains the whole sheet, so the questions over the other Products also received the changed text and are not an untreated control. 1 question is gained in every repetition (h15, a `HAVING` aggregate the agent now also selects); none is lost in every repetition.

Reading. In this two-arm, three-repetition pilot, correcting the three false statements reduced the outputs whose first rejection falls on those constructs; the admitted total moved from 58 to 67 of 120; and the statements written for the 7 questions concerned stayed at 0 of 42 admitted (1 with fences removed). Correcting these three statements alone therefore did not remove the main rejections observed here. The pilot does not bound the sheet's effect and does not separate the contributions of the sheet, of generation randomness, and of whether the questions are expressible in the fragment at all. Of the priors, the corrected arm's rejections on the removed constructs (expected 0–3 per repetition) held; the original arm was expected at 19–25 and two repetitions gave 18; the median difference was expected between 0 and $+4$ and is $+5$. The $+6$ written beforehand as the sign of a dominant sheet effect was a decision threshold for this study, not a statistical bound, and staying below it is not evidence that the effect is small. One prior was wrong in its mechanism: the two year-filter questions were expected to become admissible as date ranges, and the agent wrote typed `DATE` literals, which the fragment rejects. The fence defect was not anticipated: unparsable output, nearly all of it fenced, is the largest single rejection class of every repetition here and appeared once in the held-out run. As before, this is admission only; no statement was executed and no answer graded.

B. An LLM Agent Under Per-Query Authorization and Under FactGate

The workloads above test whether agent-written SQL is admitted. This pilot tests what an agent *does* under the two controls when it has an objective and can read refusals: run `p10-b2-agent-01` (three fresh deployments of the adversary-owner profile; 36 agent runs; results digest `0fedf0865a34`; design frozen before execution in `docs/p10_b2_agent_experiment_design.md`). The agent is an off-the-shelf headless coding assistant (claude-haiku-4-5-20251001, claude-opus-5[1m]) invoked once per step with a fixed instruction, the Product's column sheet, the objective, and the transcript so far, and it answers with one SQL statement or a final JSON; nothing in the prompt describes the fragment or the budget, and no prompt was tuned after the first run. Two arms see the same ten-row expense fixture: *RLS-only*, a PostgreSQL role under `FORCE ROW LEVEL SECURITY` scoped to the department with no cumulative accounting, and *FactGate*, the governed `query_sql` path under the owner tier ($B_R/B_D/B_O = 10/18/12$). Two objectives: *benign*, list the department's receipts of at least 1,000 with their amounts and give the department's receipt count; *probe*, the auditing rule allows only `count(*)`, recover the department's maximum single amount (truth 1910) as precisely as possible. Three samples per cell and deployment; at most twenty steps; a run ends at the agent's final answer, graded against the fixture. Table XLV reports the cells; no run failed for harness reasons.

a) Benign.: Both arms answer correctly in every run. The agent's first move is the same under both controls: count the rows, then list them all sorted by amount. RLS-only returns the listing (a median of 14 distinct cells shown, more than the objective needs); FactGate refuses that listing on the Release floor in all 9 runs at step 2, the agent reads the code and rewrites the query with the objective's own filter (`amount >= 1000`), which is admitted, and it finishes in a median of 5 steps with the root ledger at 6/10/4. The refusal cost one extra step and no correctness; the agent never saw the rows the objective did not ask for.

TABLE XLV
LLM-AGENT PILOT (36 RUNS OVER THREE DEPLOYMENTS, THREE SAMPLES PER CELL AND DEPLOYMENT): CORRECT FINALS (BENIGN: THE LISTED RECEIPTS AND THE COUNT MATCH THE FIXTURE; PROBE: AN EXACT FINAL EQUALS THE TRUTH OR AN INTERVAL FINAL CONTAINS IT, SO A CORRECT PROBE IS NOT A RECOVERED SECRET AND THE WIDTH COLUMN CARRIES THE PRECISION), MEDIAN STEPS TO THE FINAL, RUNS WITH A BUDGET REFUSAL AND THE MEDIAN STEP OF THE FIRST, TOTAL BUDGET REFUSALS, AND WHAT THE AGENT OBTAINED: FOR RLS-ONLY THE DISTINCT CELLS IT WAS SHOWN (MEDIAN), FOR FACTGATE THE ROOT LEDGER $R/D/O$ AT THE END (MEDIAN); FOR THE PROBE, THE WIDTH OF THE AGENT'S FINAL INTERVAL (MEDIAN, MINIMUM).

Objective	Arm	Correct	Steps	Refused runs / first	Refusals	Obtained	Width
benign	RLS-only	9/9	5	0 / –	0	14 cells	–
benign	FactGate	9/9	5	9 / 2	11	6/10/4	–
probe	RLS-only	9/9	20	0 / –	0	19 cells	0.4 (0)
probe	FactGate	9/9	9	9 / 7	17	4/8/11	625 (625)

b) Probe.: Under RLS-only every run recovers the maximum by threshold bisection on `count(*)`: nineteen to twenty probes, a final interval of median width 0.4 (exact in the best run), i.e., the aggregate-only rule does not protect the value. Under FactGate every run settles exactly six probes: the opening unfiltered `count(*)` charges one Outcome fact and each of the five threshold probes charges two, so the root ledger holds 11 of $B_O = 12$ and the next threshold probe, which needs two, cannot fit; every later probe is refused (the budget is not exhausted to the last fact, it is one fact short of the next charge). The agents' finals are intervals of width 625 in seven runs and 6,250 or 10,000 in two runs whose bisection started from 10^5 or 10^6 and had spent its six probes before reaching the data's range. The agent tried 17 further probes across the nine runs after the first refusal, none admitted. This is the $\lfloor B_O/2 \rfloor$ bound of the adversary study (Section XXIII-H) observed with an agent that was not told the bound, and the interval it reaches is the one the frozen bisection reaches at the same tier.

c) What this shows and does not.: One agent, one fixture, one tier: the pilot shows that a current agent reads refusal codes and re-plans, that a set budget costs a legitimate task a step rather than its answer, and that a probing task is stopped at the budget's bit bound rather than by the rule the agent was given. It does not estimate the distribution of agent behaviour, and it does not compare prompts or models.

C. Closure of the Fragment for a Governed Reporting Tier, and a Design Path

The fragment is not a subset of ad-hoc SQL chosen for convenience; it is closed under the operations a governed reporting tier performs on published Products. Such a tier serves curated Products with attested stable keys, so every join a report needs is an equi-join on those keys (connected graphs of 2–16 roles are admitted); measures are published as Product columns, so derived measures that TPC-H and SSB compute inline (`lo_extendedprice * lo_discount`) become columns or single-Product exact derived cells; reporting filters are literal comparisons on published fields; and reporting aggregation is grouping with the admitted aggregates, `HAVING`, bounded union, and pagination. Under these operations the fragment is closed: every term built from admitted terms by an admitted operator is again admitted, and every such term has a total Dependency rule. What the fragment excludes is what a reporting tier delegates upstream to ETL (arbitrary expressions, subqueries, outer joins that materialize absence) or forbids (comparisons of one Product's cell with another's).

Three excluded constructs dominate the rejections of all workloads and each has a rule sketch consistent with the algebra:

- **`OR` (disjunctive witnesses).** $b_1 \vee b_2$ keeps a row when either disjunct is T; its Dependency footprint is the predicate cells of every *satisfied* disjunct (positive-output rule), merged by alternative union/max, and its Outcome atoms are those of both disjuncts plus a composite binding the disjunction. Conservation follows the union-distinct argument: one query per disjunct settles the same support.
- **Non-literal comparisons (two-cell atoms).** $a\ op\ b$ over two fields reads both cells, so both enter the footprint; the Outcome atom binds both field identities and no literal, so varying the compared pair is a new atom while re-spellings are not.
- **Outer joins (null-extended rows).** A preserved row without a match carries the preserved side's support only; null-extended cells release nothing and depend on nothing, and the row identity binds the absence of a match so that a later inner-join re-issue is a distinct Outcome.

Each sketch keeps $\mathrm{support}(W) = S$, the representation-closure induction, and the base-ledger conservation argument; none is implemented or evaluated here.

XXXII. ENTERPRISE INTEGRATION AND GOVERNANCE VISION

This section describes a brownfield-compatible, incremental deployment vision, not an evaluated production deployment. FactGate preserves existing enterprise systems and places a shared logical data-egress contract after curated products at the governed serving tier. Table XLVIII separates the inputs FactGate consumes, the contract checks it adds, and responsibilities that remain with each system and its operators.

TABLE XLVI
REJECTED CONSTRUCTS ACROSS THE TPC-H, SSB, AND AGENT-WRITTEN WORKLOADS: WHETHER ADMITTING THEM IS AN ENGINEERING GAP (EXISTING DEPENDENCY RULE APPLIES) OR NEEDS A NEW RULE.

Construct	Class	Dependency rule
scalar functions in projections	engineering (per function)	the argument columns; only attested deterministic functions
division / multi-source arithmetic	new rule	the single-Product exact $\{+, -, \times\}$ case is admitted as derived cells; division's scale semantics and cross-source expressions remain open
comparison to a column, expression, or subquery	new rule	two-cell predicate atoms; a scalar subquery is a nested effect
`OR` in `WHERE`	new rule	disjunctive witnesses: predicate-cell facts per satisfied disjunct
`BETWEEN` over text	new rule	collation-dependent range atoms
outer joins	new rule	null-extended rows have no matching witness
subqueries, `LIMIT` over joins	new rule	nested effects and multi-product page order

TABLE XLVII
FIRST REJECTION REASON OF EACH AGENT-WRITTEN STATEMENT.

Construct	Reason code	Count	Statements
scalar function in the projection or aggregate	AGGREGATE_UNSUPPORTED	3	Q06, Q14, Q40
non-literal comparison (column or subquery)	FILTER_LITERAL_UNSUPPORTED	2	Q33, Q39
arithmetic in the projection	PROJECTION_EXPRESSION_UNSUPPORTED	2	Q09, Q20
LEFT JOIN	LEFT_JOIN_UNSUPPORTED	1	Q24
OR disjunction in WHERE	BOOLEAN_OPERATOR_UNSUPPORTED	1	Q31
BETWEEN / LIKE predicate	FILTER_PREDICATE_UNSUPPORTED	1	Q10
HAVING compared to an expression	HAVING_LITERAL_UNSUPPORTED	1	Q19
LIMIT over a multi-product join	PAGINATION_UNSUPPORTED	1	Q27

Illustrative scenarios (not evaluated as production deployments). In a telecommunications enterprise, headquarters marketing could analyze nationwide aggregate metrics while a provincial marketer's mandatory regional scope limits the same Product to one province. Subscriber phone numbers, names, and government identifiers would not enter the marketing Product. Where small-cohort disclosure matters, a separate policy, such as a minimum-contributor threshold, statistical disclosure control, differential privacy, or DFC, remains necessary. An enterprise-wide salary query would require a separate HR-leader-approved task. Operations personnel could use a distinct equipment-and-facility Product, but the present analytical FactGate path would not control a network in real time. In banking, branch staff could analyze regional aggregates, while risk or fraud casework uses a short-lived, high-sensitivity, independently approved task. This does not make FactGate a regulatory-compliance decision maker. In an electric utility, regional load, loss, and equipment-health Products could remain separate from marketing and finance domains; FactGate would not perform subsecond control, protection actions, or equipment-configuration writes. The scenarios only illustrate how Product, scope, approval, and the task-bound root-family Exposure Ledger compose.

Future Policy Studio (not implemented). A future governance plane could move a profile through Draft $\rightarrow$ metadata discovery $\rightarrow$ stable-key validation $\rightarrow$ sensitive-field inheritance $\rightarrow$ generated negative tests $\rightarrow$ shadow execution $\rightarrow$ multi-role approval $\rightarrow$ signed profile publication. Data owners would decide purpose and releasable scope; security/privacy teams would define sensitive boundaries; IAM/OA would attest identity and approval; DBAs/data engineers would validate database facts and performance mappings; FactGate would compile and enforce the published contract; and auditors would reconcile approval, settlement, and AVAILABLE evidence. Multi-engine connectors, DFC/Passant composition, principal/tenant-wide policy, differential privacy, and cross-tool IFC remain separate future work. The present prototype does not implement this complete governance platform.

TABLE XLVIII
ENTERPRISE INTERFACE DIVISION. THESE ARE ARCHITECTURAL BOUNDARIES, NOT PRODUCTION-INTEGRATION EVIDENCE.

Existing system	Supplies to FactGate	FactGate validates or adds	Outside FactGate responsibility
Lake / warehouse	Curated snapshot or governed serving Product	Immutable publication and version binding	Raw-lake access, ETL, source quality, and stewardship
Data Catalog	Product, schema, owner, lineage, classification, and candidate business keys	Structural and version binding of attested stable identities and the signed Product, field, profile, and supported-semantics contract	Business ownership, classification, and business-key stability
IAM / HR	Authenticated subject, department, region, and role	Task-subject and mandatory organizational-scope binding	Identity lifecycle and attribute correctness
OA / BPM	Purpose, approver, expiry, chain, and decision	Immutable signed Grant	Approval appropriateness and workflow design
Object store	Private staging, canonical storage, and access control	Hash-bound intent and PENDING/AVAILABLE protocol	Durability, key custody, and storage operations
Audit / SIEM	Enterprise evidence destination	Settlement, receipt, and availability evidence	Correlation, retention, and incident response
Agent platform	Plans, retries, pages, delegation, and tool calls	Untrusted-client enforcement boundary	Model/prompt behavior and cross-tool information flow